Artificial Intelligence & IoT

Constraint Satisfaction Problems



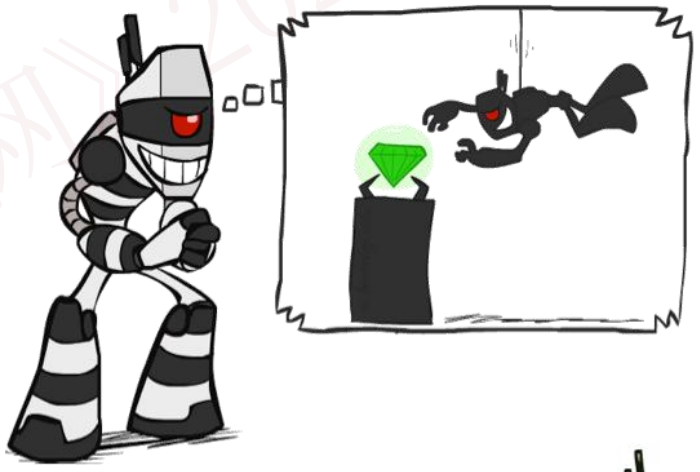


Instructor: Kaikai Pan
Zhejiang University

[These slides were adapted from the slides by Dan Klein and Pieter Abbeel for CS188 Intro to AI at UC Berkeley. http://ai.berkeley.edu.]

What is Search For?

- Assumptions about the world: a single agent, deterministic actions, fully observed state, discrete state space

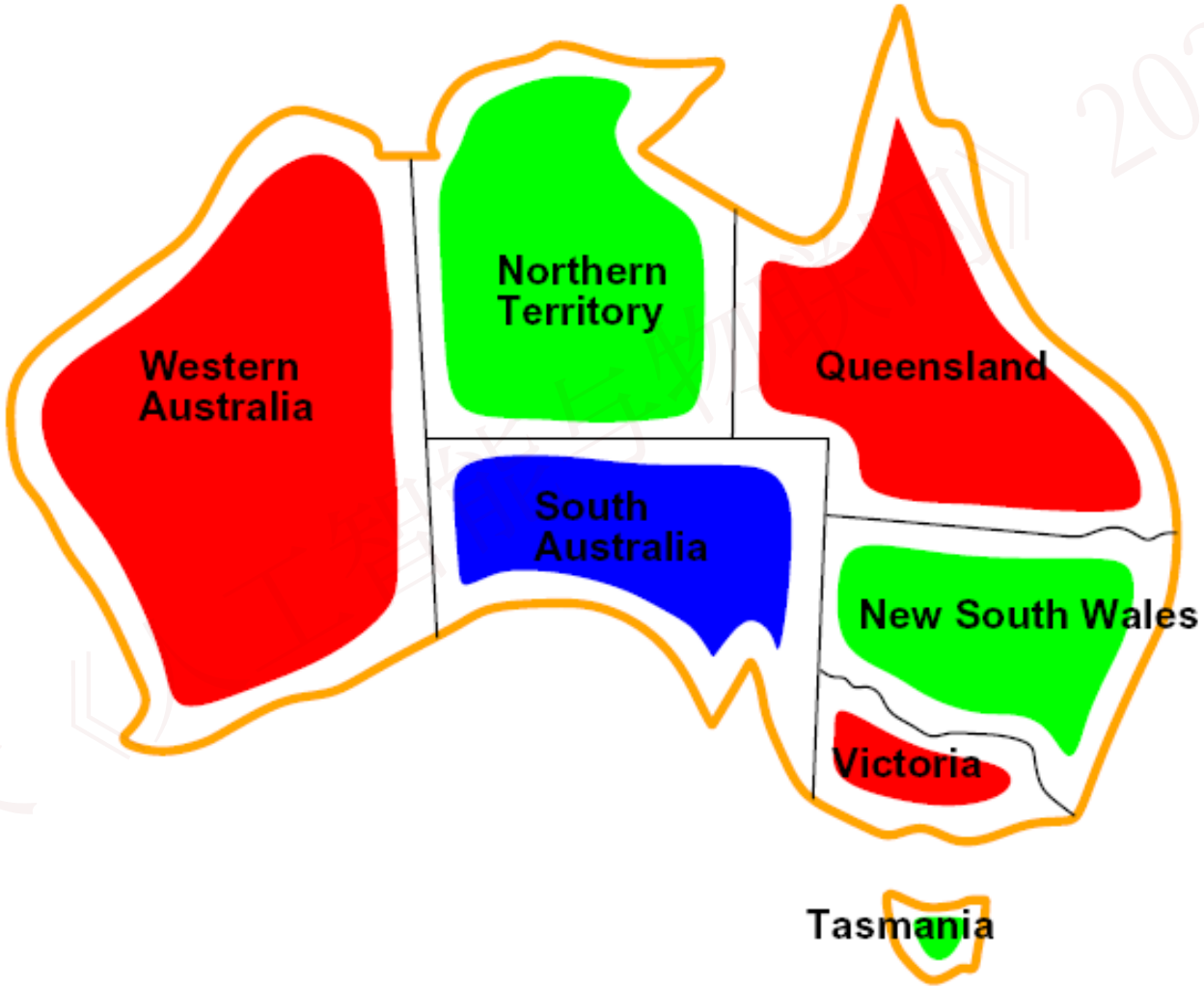


- Planning (计划/规划): sequences of actions
  - The path to the goal is the important thing
  - Paths have various costs, depths
  - Heuristics give problem-specific guidance

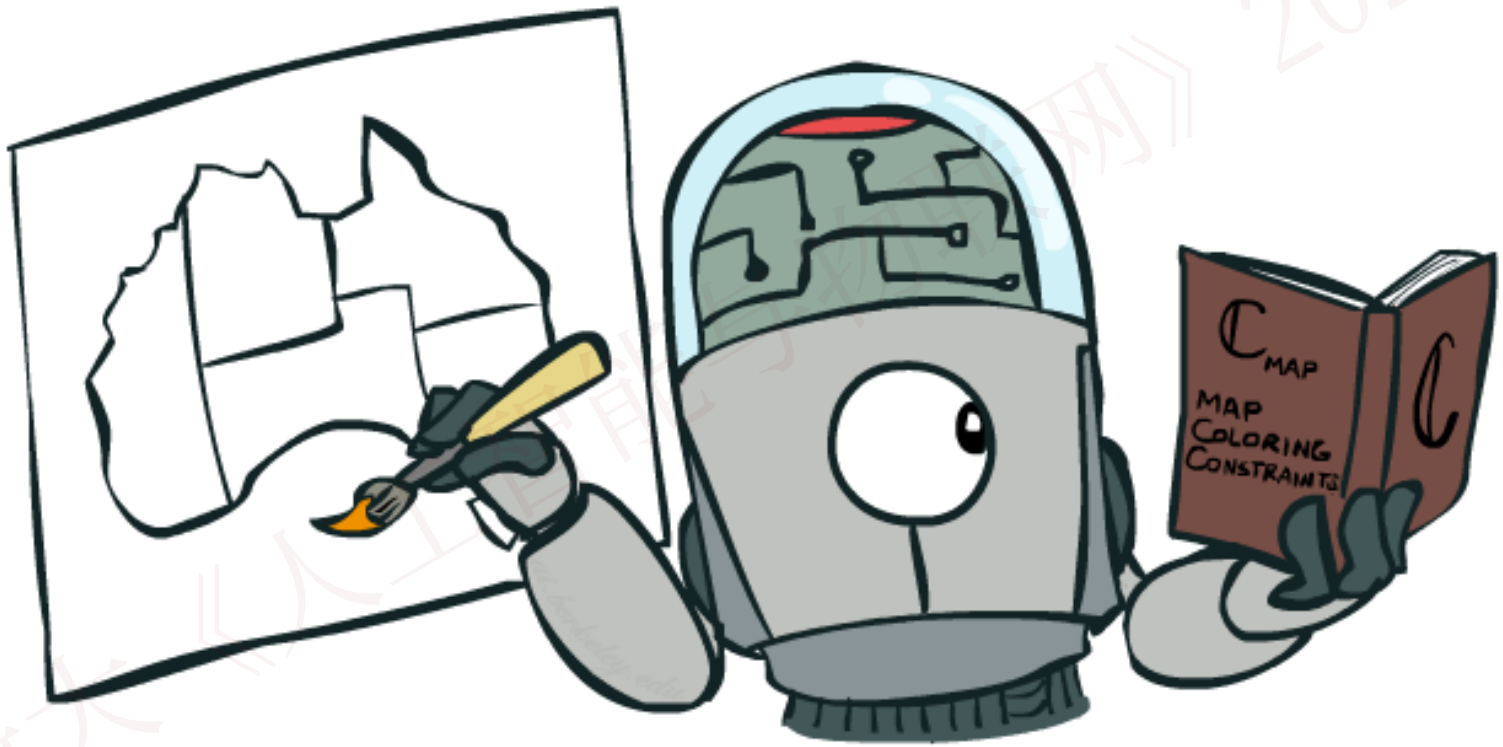
- Identification (鉴定/认定): assignments (赋值) to variables
  - The goal itself is important, not the path
  - CSPs are specialized for identification problems



CSP Examples



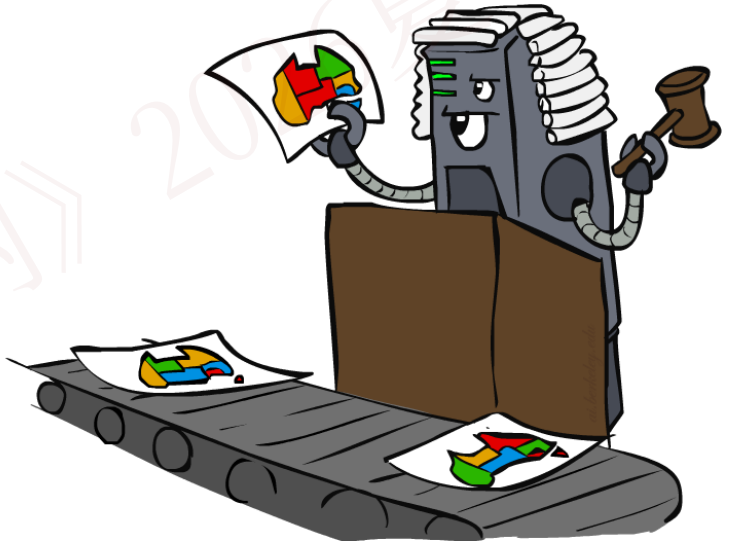
Constraint Satisfaction Problems (约束满足问题)



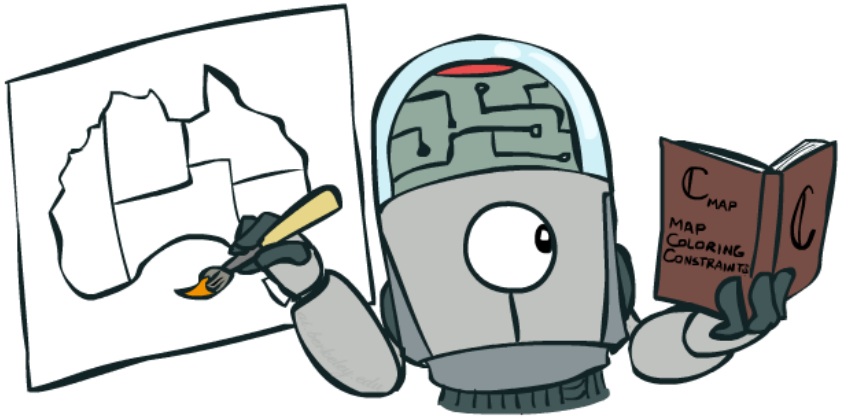
浙大《人工智能与机器学习》2026夏

Constraint Satisfaction Problems

- Standard search problems:
  - State can be arbitrary data structure
  - Goal test can be any function over states
  - Successor function can also be anything



- Constraint satisfaction problems (CSPs):
  - A special subset of search problems
  - State is defined by variables (变量) X_i with values from a domain (值域) D (sometimes D depends on i)
  - Goal test is a set of constraints specifying allowable combinations of values for subsets of variables



- Allows useful general-purpose algorithms with more power than standard search algorithms

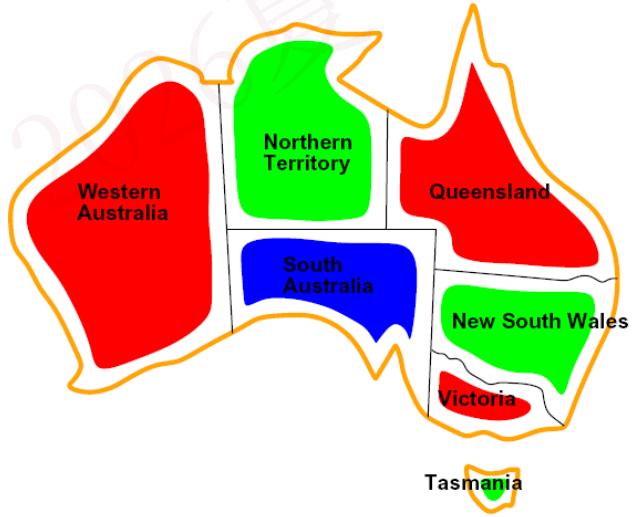
Example: Map Coloring

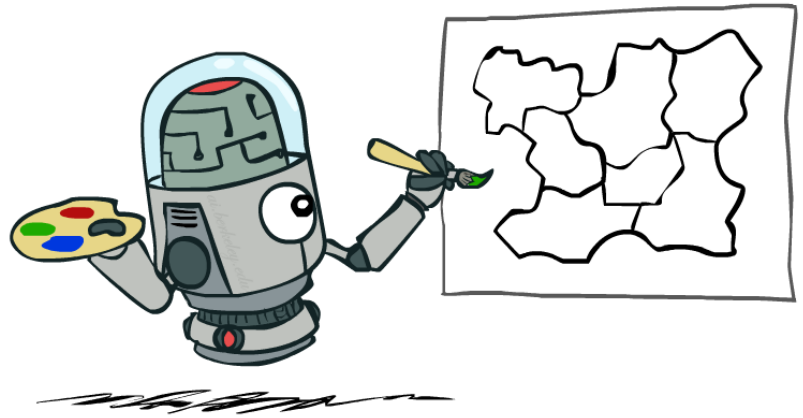
- Variables: $\mathrm{WA},\ \mathrm{NT},\ \mathrm{Q},\ \mathrm{NSW},\ \mathrm{V},\ \mathrm{SA},\ \mathrm{T}$
- Domains: $\mathrm{D} = \{\mathrm{red}, \mathrm{green}, \mathrm{blue}\}$
- Constraints: adjacent regions must have different colors

  Implicit: $\mathrm{WA} \neq \mathrm{NT}$

  Explicit: $(\mathrm{WA}, \mathrm{NT}) \in \{(\mathrm{red}, \mathrm{green}), (\mathrm{red}, \mathrm{blue}), \ldots\}$
- Solutions (解) are assignments satisfying all constraints, e.g.:

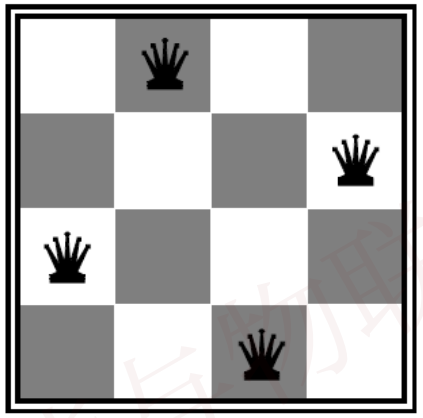
  $\{\mathrm{WA}{=}\mathrm{red},\ \mathrm{NT}{=}\mathrm{green},\ \mathrm{Q}{=}\mathrm{red},\ \mathrm{NSW}{=}\mathrm{green},$
  $\mathrm{V}{=}\mathrm{red},\ \mathrm{SA}{=}\mathrm{blue},\ \mathrm{T}{=}\mathrm{green}\}$

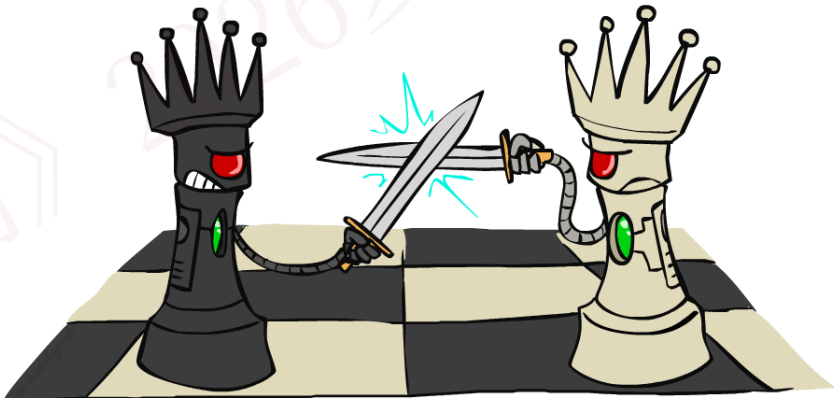




Example: N-Queens (N-皇后)

- Formulation (形式化表示) 1:
  - Variables: X_{ij}
  - Domains: $\{0, 1\}$
  - Constraints





$$\forall i, j, k \quad (X_{ij}, X_{ik}) \in \{(0,0), (0,1), (1,0)\}$$
$$\forall i, j, k \quad (X_{ij}, X_{kj}) \in \{(0,0), (0,1), (1,0)\}$$
$$\forall i, j, k \quad (X_{ij}, X_{i+k,j+k}) \in \{(0,0), (0,1), (1,0)\}$$
$$\forall i, j, k \quad (X_{ij}, X_{i+k,j-k}) \in \{(0,0), (0,1), (1,0)\}$$

$$\sum_{i,j} X_{ij} = N$$

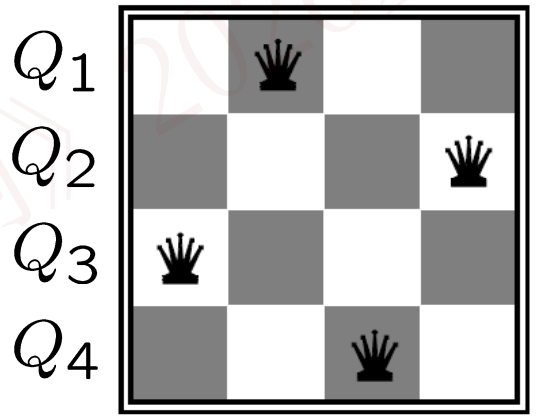
Example: N-Queens

- Formulation 2:
  - Variables: Q_k
  - Domains: $\{1, 2, 3, \ldots N\}$
  - Constraints:

Implicit (隐式): $\forall i, j$ non-threatening(Q_i, Q_j)

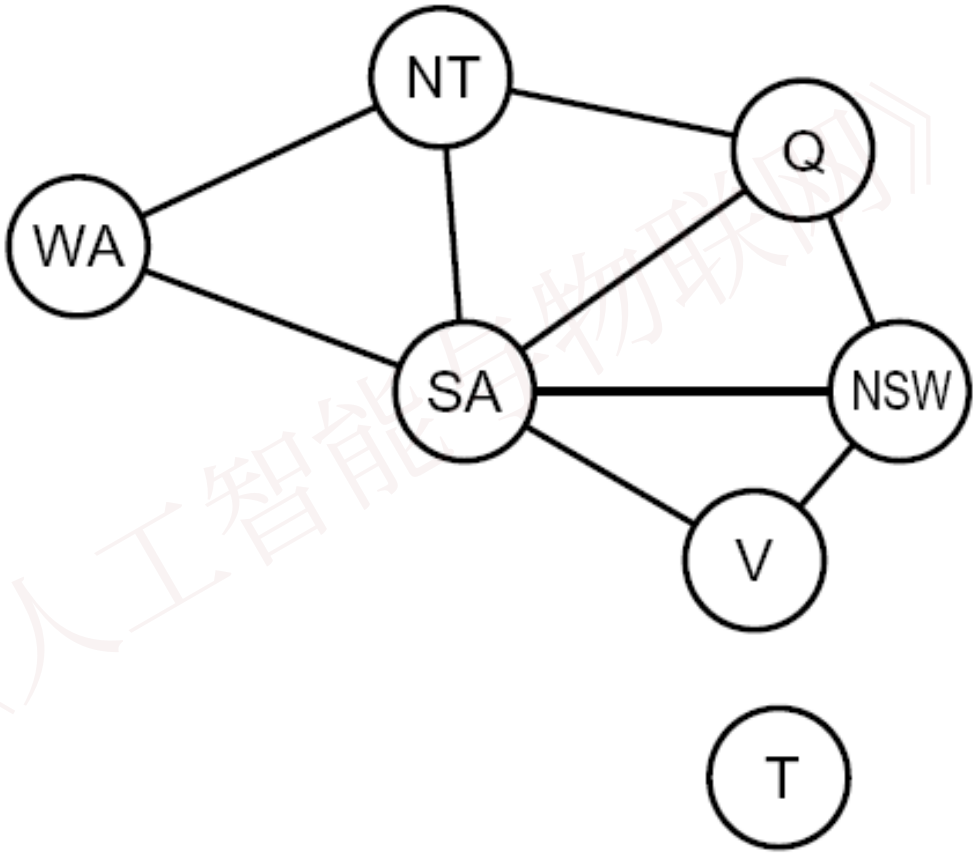
Explicit (显式): $(Q_1, Q_2) \in \{(1,3), (1,4), \ldots\}$

$\cdots$



浙大《人工智能与物联网》2026夏

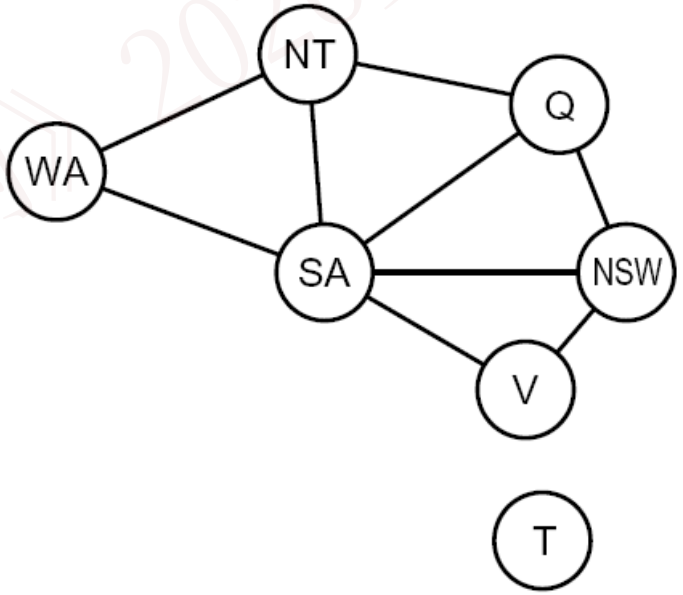
Constraint Graphs (约束图)



浙大《人工智能与物联网》2026夏

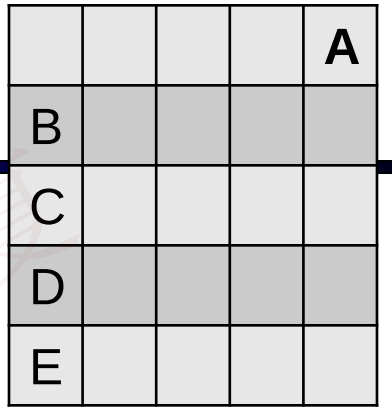
Constraint Graphs

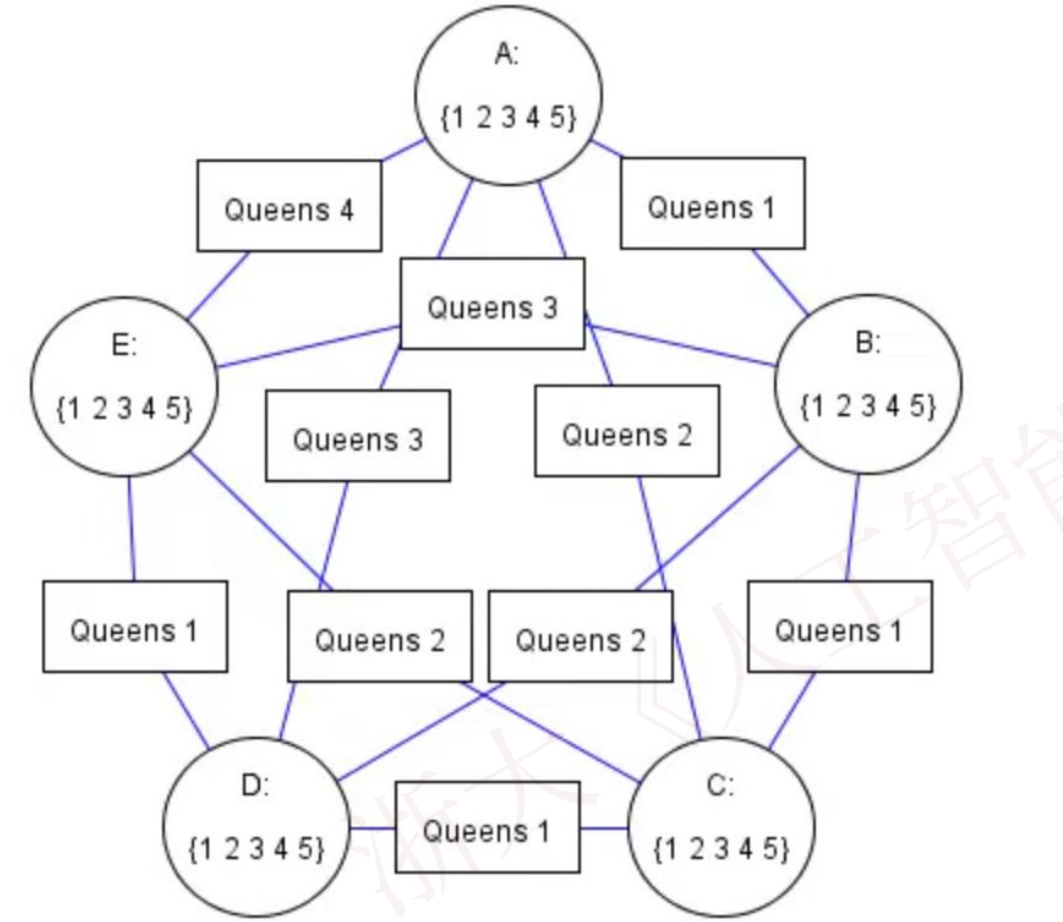
- Binary CSP: each constraint relates (at most) two variables
- Binary constraint graph (二元约束图): nodes are variables, edges show constraints
- General-purpose CSP algorithms use the graph structure to speed up search. E.g., Tasmania is an independent subproblem!



浙大人工智能与物联网 2026夏

N-Queens





- Queen 1, e.g., A & B: (1,3) (1,4) (1,5) (2,4) (2,5) (3,1) (3,5) (4,1) (4,2) (5,1) (5,2)
- Queen 2, e.g., A & C: (1,2) (1,4) (1,5) (2,1) (2,3) (2,5) (3,2) (3,4) (4,1) (4,3) (4,5) (5,1) (5,2) (5,4)
- Queen 3, e.g., A & D: (1,2) (1,3) (1,5) (2,1) (2,3) (2,4) (3,1) (3,2) (3,4) (3,5) (4,2) (4,3) (4,5) (5,1) (5,3) (5,4)
- Queen 4, e.g., A & E: (1,2) (1,3) (1,4) (2,1) (2,3) (2,4) (2,5) (3,1) (3,2) (3,4) (3,5) (4,1) (4,2) (4,3) (4,5) (5,2) (5,3) (5,4)

Example: Cryptarithmetic

- Variables:

  $F\ T\ U\ W\ R\ O\ X_1\ X_2\ X_3$

- Domains:

  $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

- Constraints:

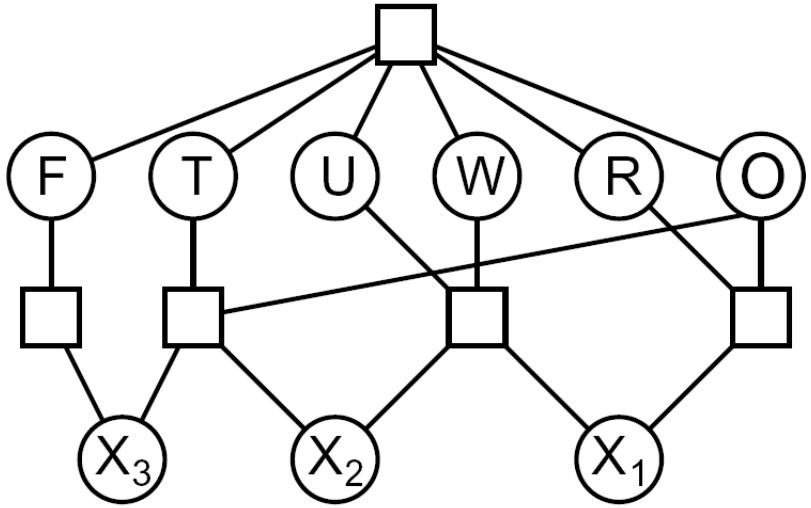
  alldiff(F, T, U, W, R, O)

  $O + O = R + 10 \cdot X_1$

  $\ldots$

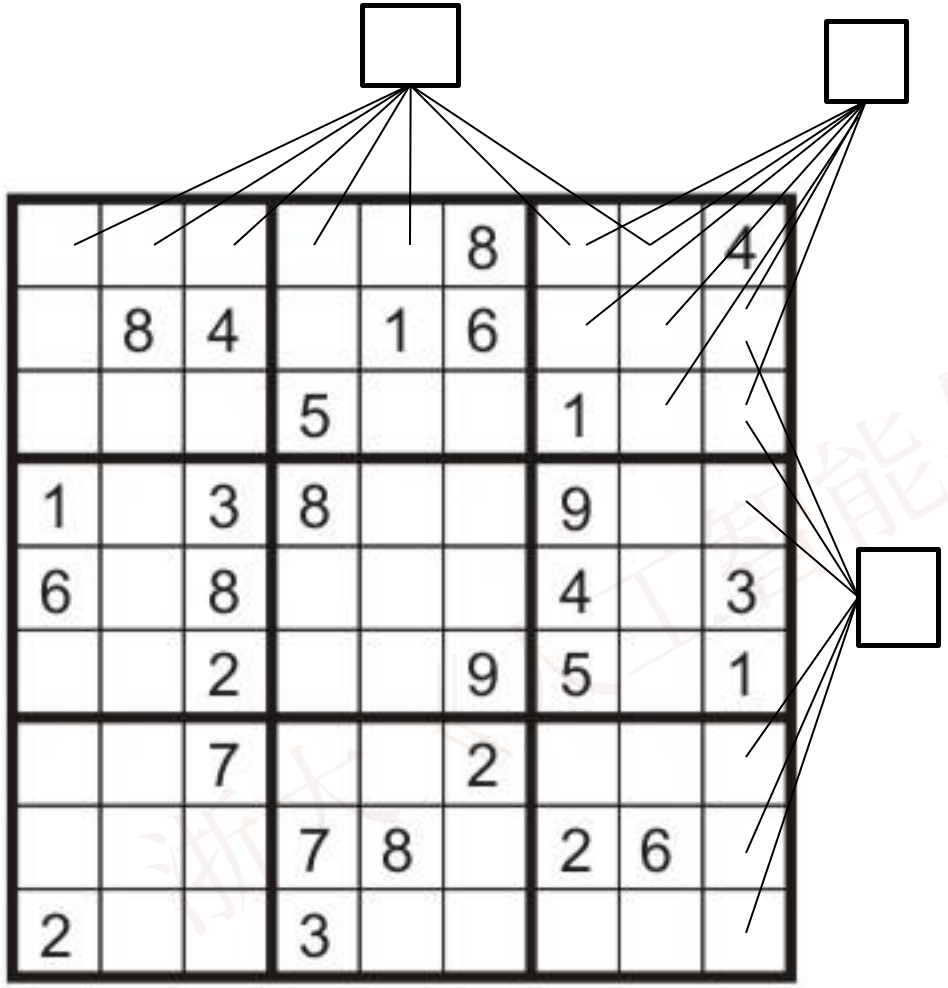
```
  T W O
+ T W O
-------
F O U R
```





浙大《人工智能与物联网》2026夏

Example: Sudoku



- Variables:
  - Each (open) square
- Domains:
  - {1,2,…,9}
- Constraints:

  9-way alldiff for each column

  9-way alldiff for each row

  9-way alldiff for each region

Varieties of CSPs and Constraints



Varieties of CSPs

- Discrete Variables (离散变量)
  - Finite domains (有限值域)
    - Size d means $O(d^n)$ complete assignments
  - Infinite domains (integers, strings, etc.) (无限值域)
    - E.g., job scheduling, variables are start/end times for each job



- Continuous variables (连续变量)
  - E.g., start/end times for Hubble Telescope observations



浙大《人工智能与物联网》2026夏

Varieties of Constraints

- Varieties of Constraints
  - Unary constraints (一元约束) involve a single variable (equivalent to reducing domains), e.g.:

$$SA \neq green$$

  - Binary constraints (二元约束) involve pairs of variables, e.g.:

$$SA \neq WA$$

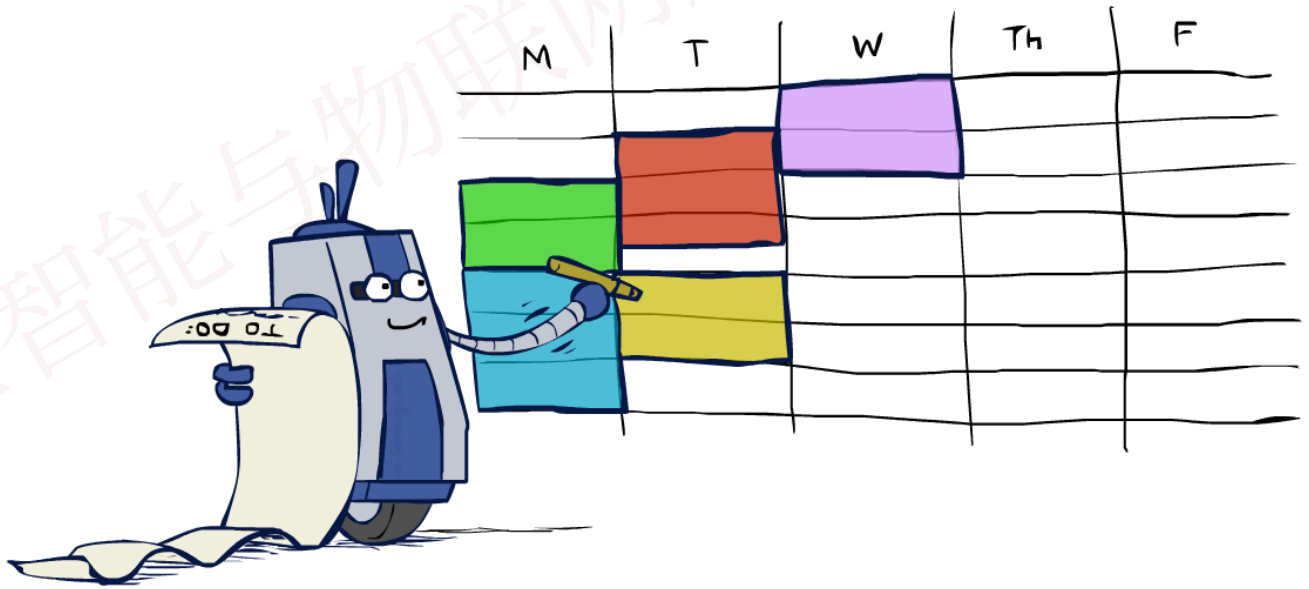
  - Higher-order constraints involve 3 or more variables:
    e.g., cryptarithmetic column constraints



- Preferences (soft constraints 偏好约束):
  - E.g., red is better than green
  - Often representable by a cost for each variable assignment

Real-World CSPs

- Assignment problems: e.g., who teaches what class
- Timetabling problems: e.g., which class is offered when and where?
- Hardware configuration
- Transportation scheduling
- Factory scheduling
- Circuit layout
- Fault diagnosis
- … lots more!



- Many real-world problems involve real-valued variables…

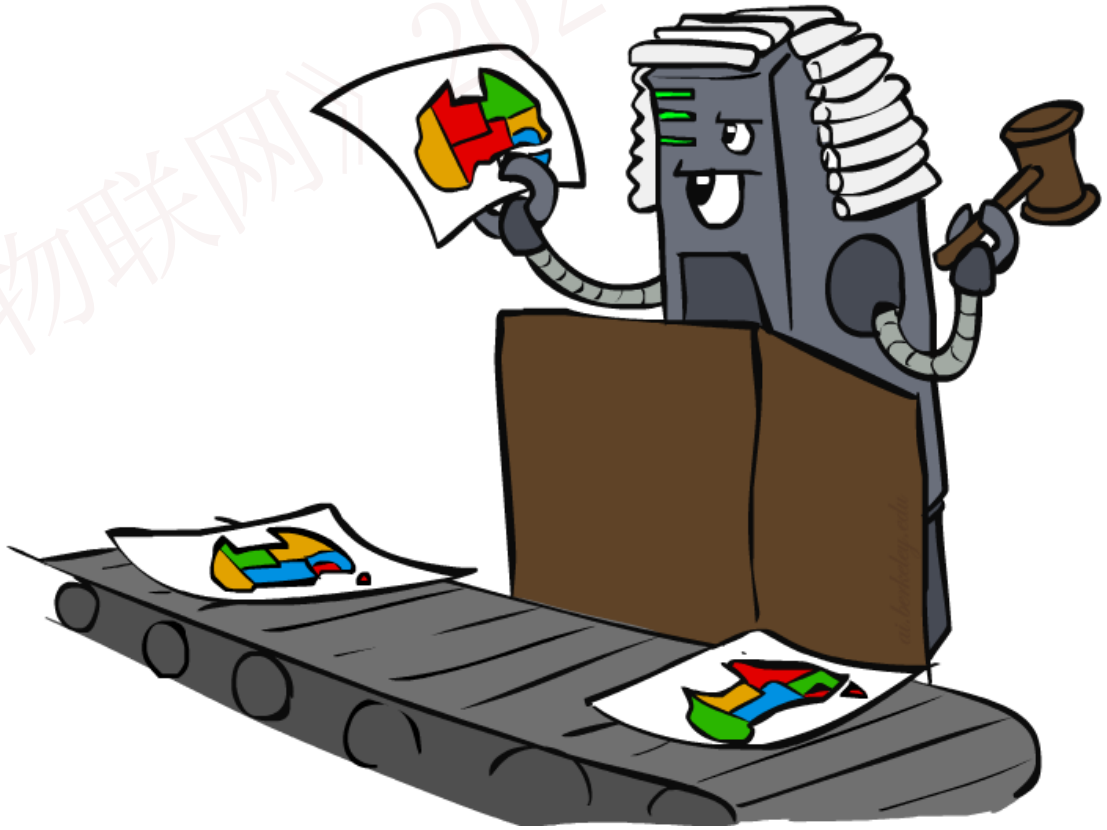
浙大《人工智能与物联网》2026夏

Solving CSPs



Standard Search Formulation

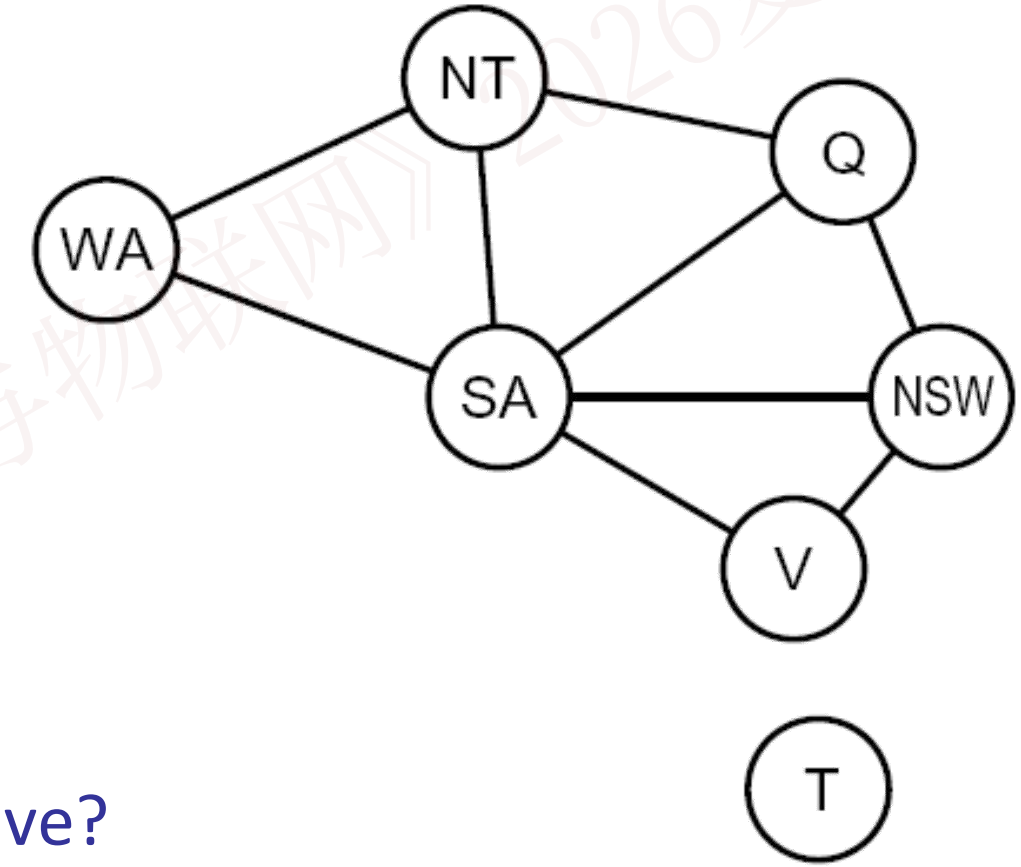
- Standard search formulation of CSPs
- States defined by the values assigned so far (partial assignments 部分赋值)
  - Initial state: the empty assignment, {}
  - Successor function: assign a value to an unassigned variable
  - Goal test: the current assignment is complete (完备) and satisfies all constraints
- We'll start with the straightforward, naïve approach, then improve it



浙大《人工智能与物联网》2026夏

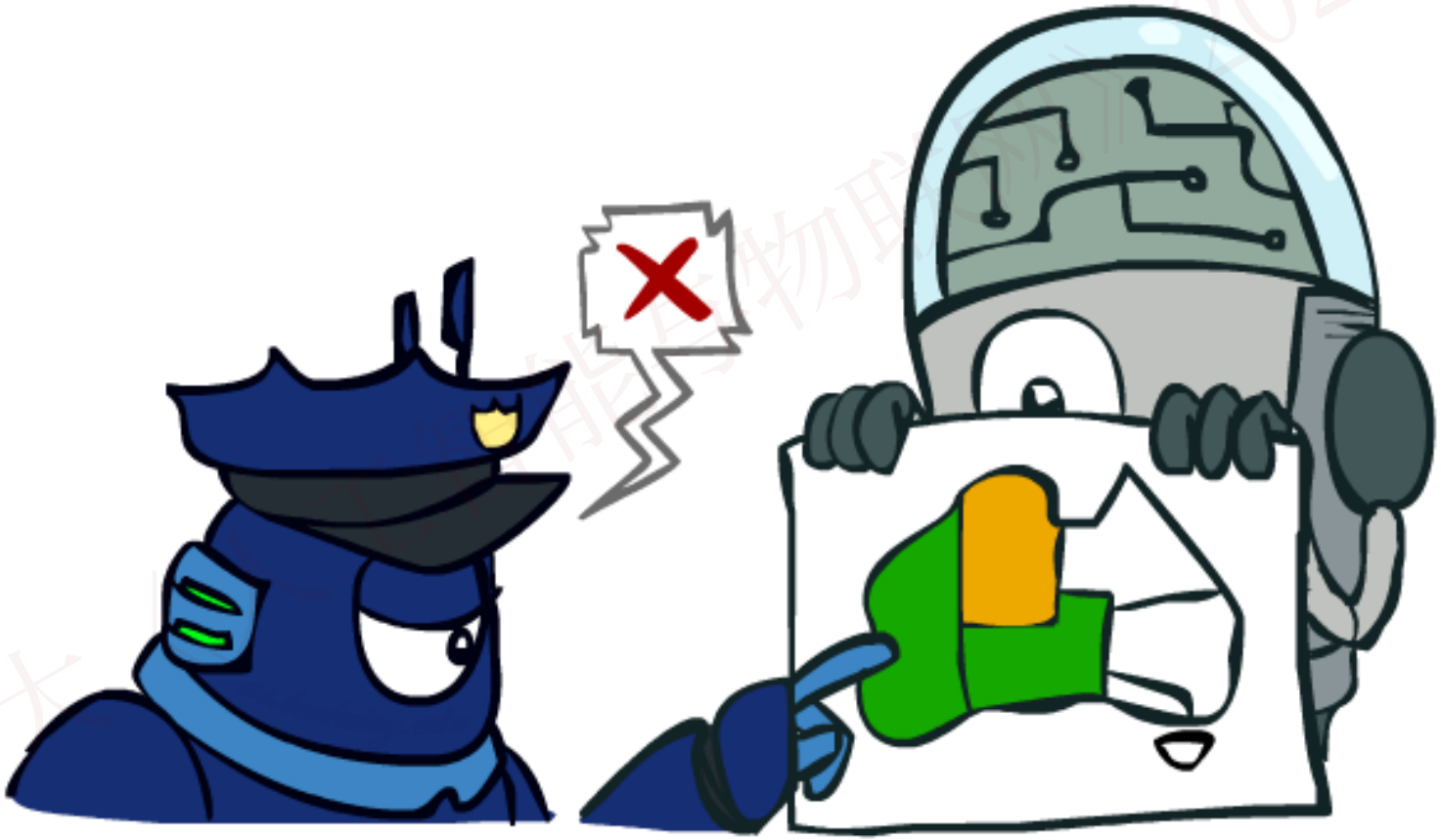
Search Methods

- What would BFS do?
- What would DFS do?
- What problems does naïve search have?



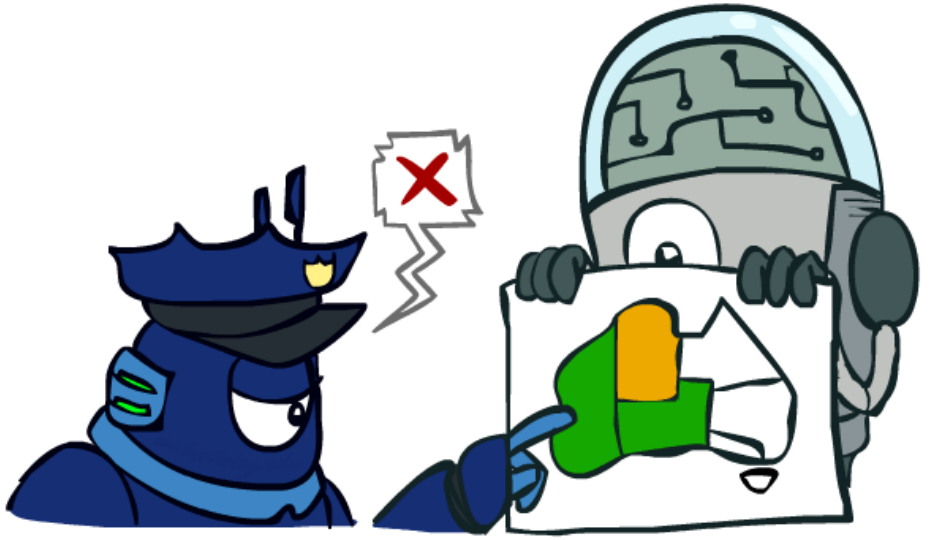
浙大《人工智能与物联网》2026夏

Backtracking Search (回溯搜索)

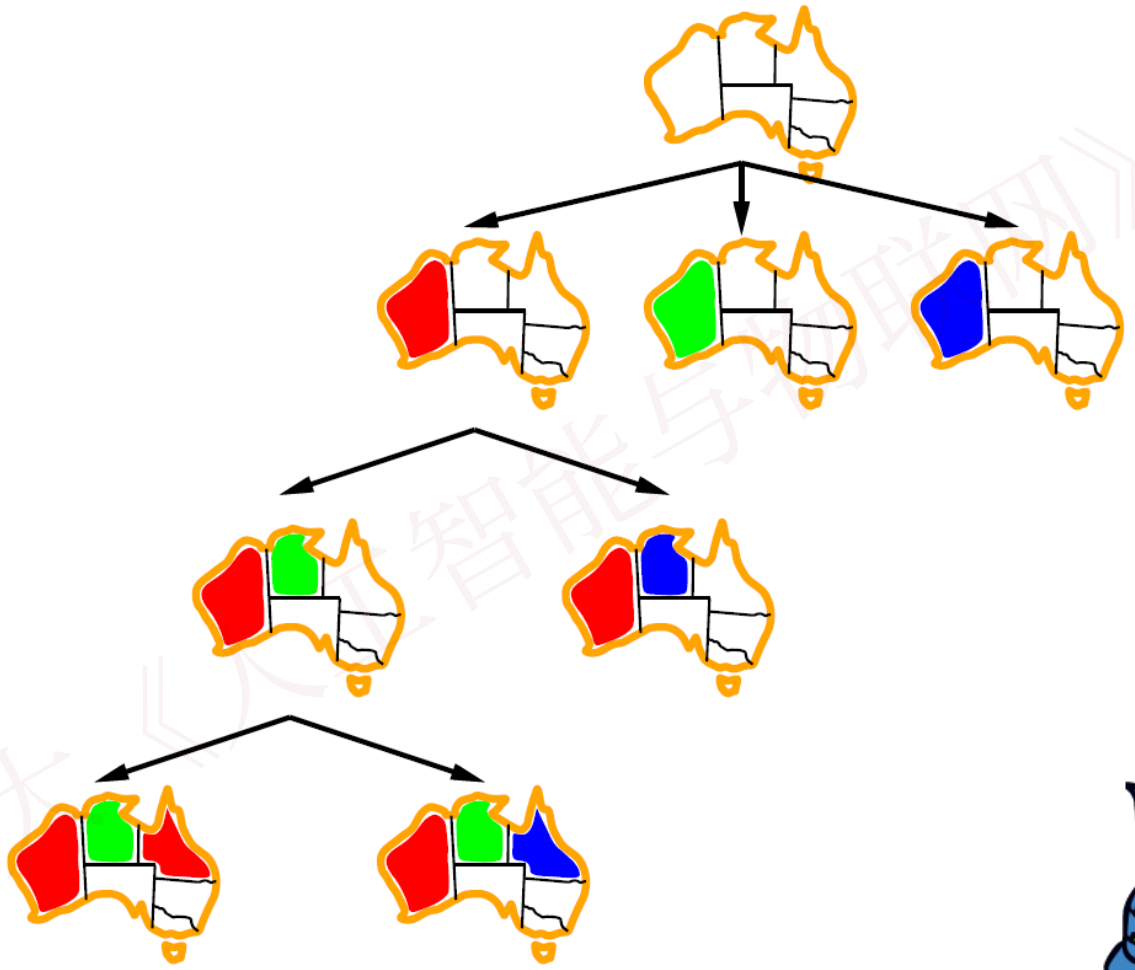


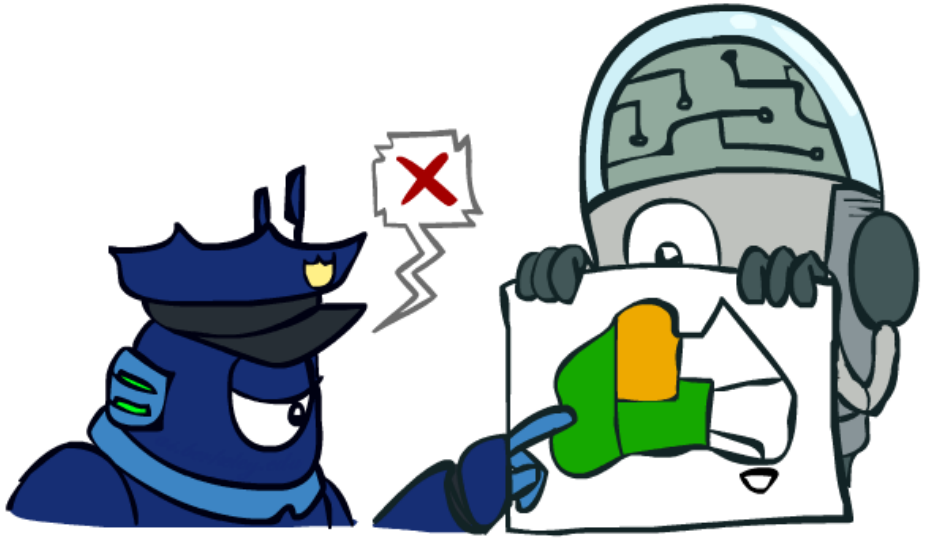
Backtracking Search (回溯搜索)

- Backtracking search is the basic uninformed algorithm for solving CSPs
- Idea 1: One variable at a time
  - Variable assignments are commutative (可交换性), so fix ordering (固定顺序)
  - I.e., [WA = red then NT = green] same as [NT = green then WA = red]
  - Only need to consider assignments to a single variable at each step
- Idea 2: Check constraints as you go
  - I.e. consider only values which do not conflict (冲突，不相容) previous assignments
  - Might have to do some computation to check the constraints
  - "Incremental goal test"
- Depth-first search with these two improvements is called *backtracking search*
- Can solve n-queens for n ≈ 25



Backtracking Example





浙大《人工智能导论(研)》2026夏

Backtracking Search

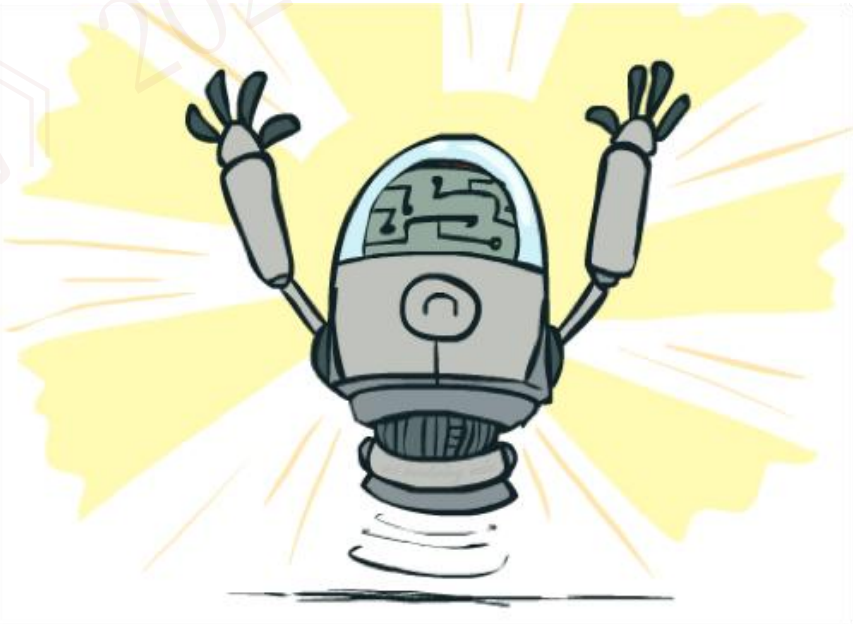
```
function BACKTRACKING-SEARCH(csp) returns solution/failure
    return RECURSIVE-BACKTRACKING({ }, csp)

function RECURSIVE-BACKTRACKING(assignment, csp) returns soln/failure
    if assignment is complete then return assignment
    var ← SELECT-UNASSIGNED-VARIABLE(VARIABLES[csp], assignment, csp)
    for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
        if value is consistent with assignment given CONSTRAINTS[csp] then
            add {var = value} to assignment
            result ← RECURSIVE-BACKTRACKING(assignment, csp)
            if result ≠ failure then return result
            remove {var = value} from assignment
    return failure
```

- Backtracking = DFS + variable-ordering + fail-on-violation
- What are the choice points?

Improving Backtracking

- General-purpose ideas give huge gains in speed
- Filtering (筛选): Can we detect inevitable failure early?
- Ordering:
  - Which variable should be assigned next?
  - In what order should its values be tried?
- Structure: Can we exploit the problem structure?



浙大《人工智能与物联网》2026夏

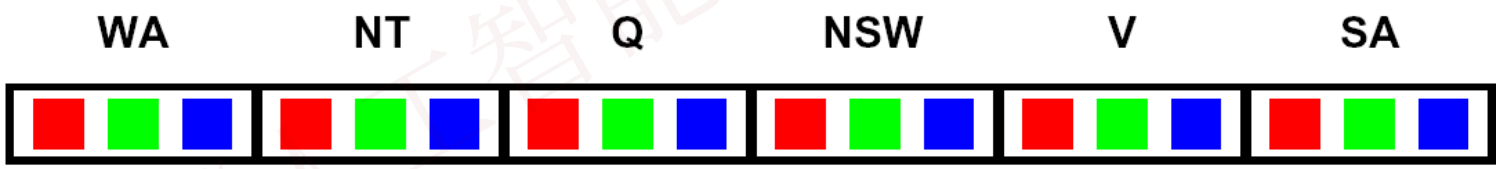
Filtering



Filtering: Forward Checking (前向检验)

- Filtering: Keep track of domains for unassigned variables and remove bad options
- Forward checking: Remove values that violate a constraint when added to the existing assignment

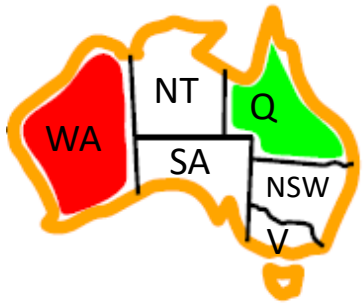




浙大《人工智能与物联网》2020春

Filtering: Constraint Propagation (约束传播)

- Forward checking propagates information from assigned to unassigned variables, but doesn't provide early detection for all failures:

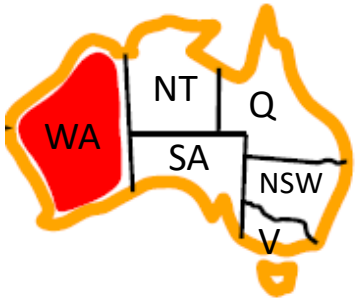


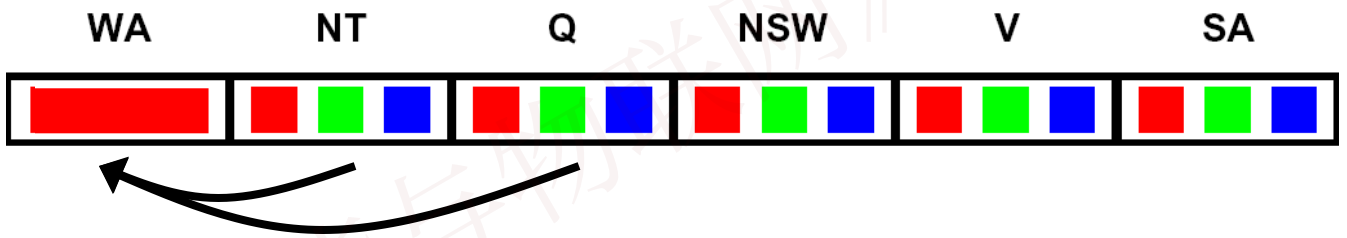


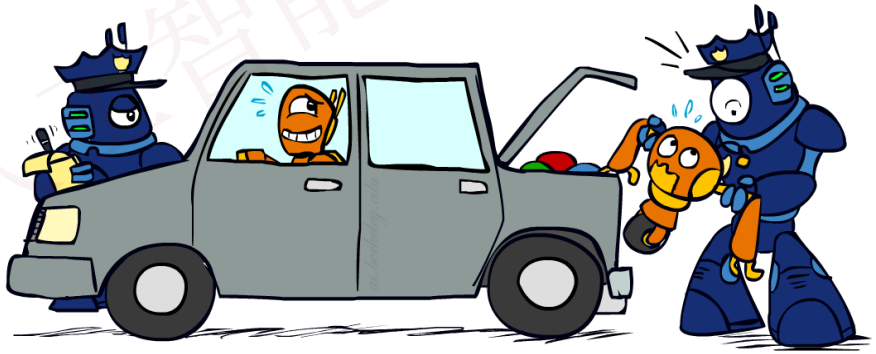
- NT and SA cannot both be blue!
- Why didn’t we detect this yet?
- *Constraint propagation:* reason from constraint to constraint (从一个约束推理到另一个约束)

Consistency of A Single Arc (弧相容)

- An arc X → Y is consistent iff for *every* x in the tail (尾部，直线端) there is *some* y in the head (头部，箭头端) which could be assigned without violating a constraint





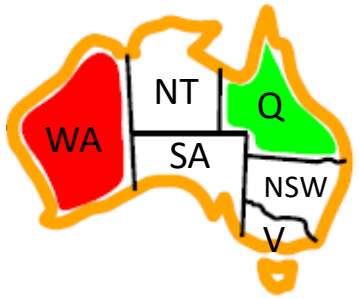


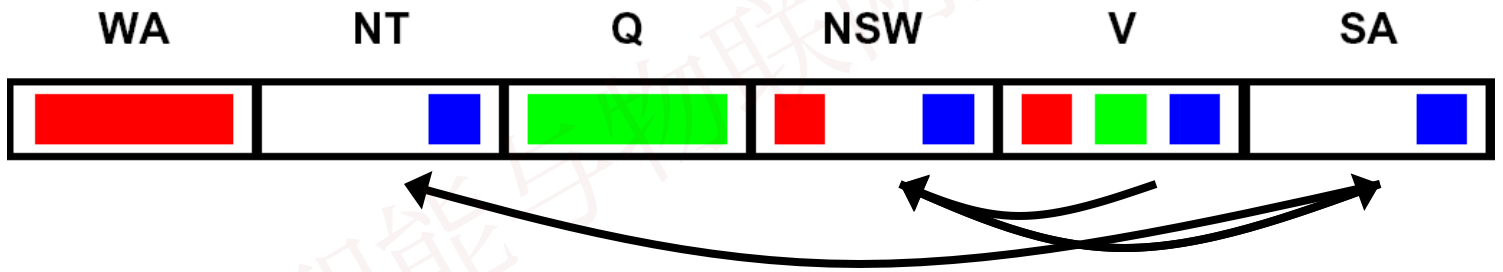
Delete from the tail!

- Forward checking: Enforcing consistency of arcs pointing to each new assignment

Arc Consistency of an Entire CSP

- A simple form of propagation makes sure all arcs are consistent:





- Important: If X loses a value, neighbors of X need to be rechecked!
- Arc consistency detects failure earlier than forward checking
- Can be run before or after each assignment
- What's the disadvantage of enforcing arc consistency?

Remember:
Delete from the tail (直线端)!

浙大《人工智能导论》2026夏

Enforcing Arc Consistency in a CSP

```
function AC-3( csp) returns the CSP, possibly with reduced domains
  inputs: csp, a binary CSP with variables {X_1, X_2, ..., X_n}
  local variables: queue, a queue of arcs, initially all the arcs in csp

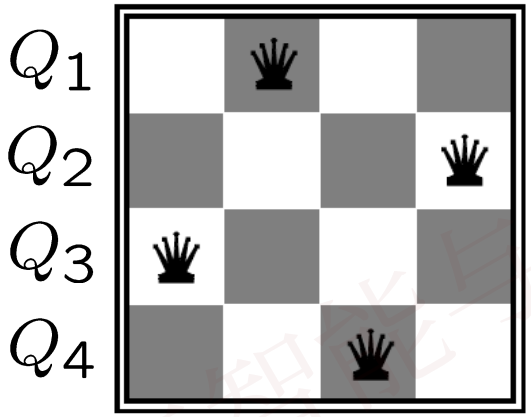
  while queue is not empty do
    (X_i, X_j) ← REMOVE-FIRST(queue)
    if REMOVE-INCONSISTENT-VALUES(X_i, X_j) then
      for each X_k in NEIGHBORS[X_i] do
        add (X_k, X_i) to queue

---

function REMOVE-INCONSISTENT-VALUES( X_i, X_j) returns true iff succeeds
  removed ← false
  for each x in DOMAIN[X_i] do
    if no value y in DOMAIN[X_j] allows (x,y) to satisfy the constraint X_i ↔ X_j
      then delete x from DOMAIN[X_i]; removed ← true
  return removed
```

- Runtime: $O(n^2 d^3)$

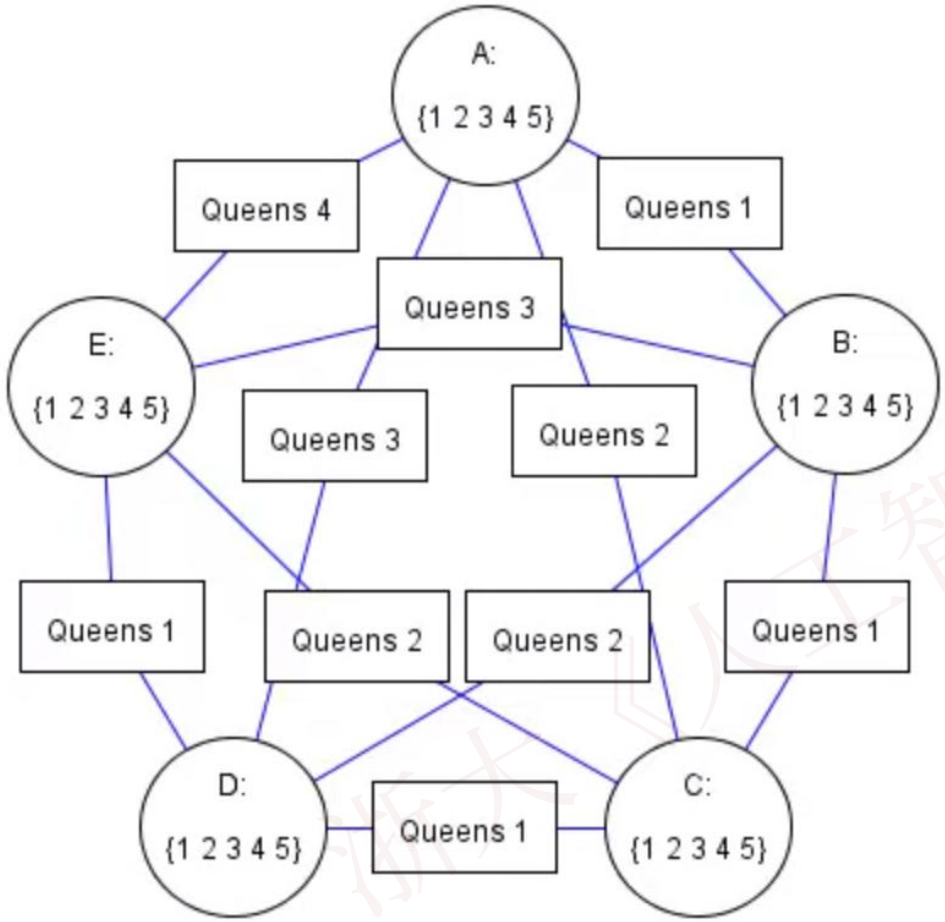
Enforcing Arc Consistency: 4-Queens



浙大《人工智能与物联网》2026夏

5-Queens

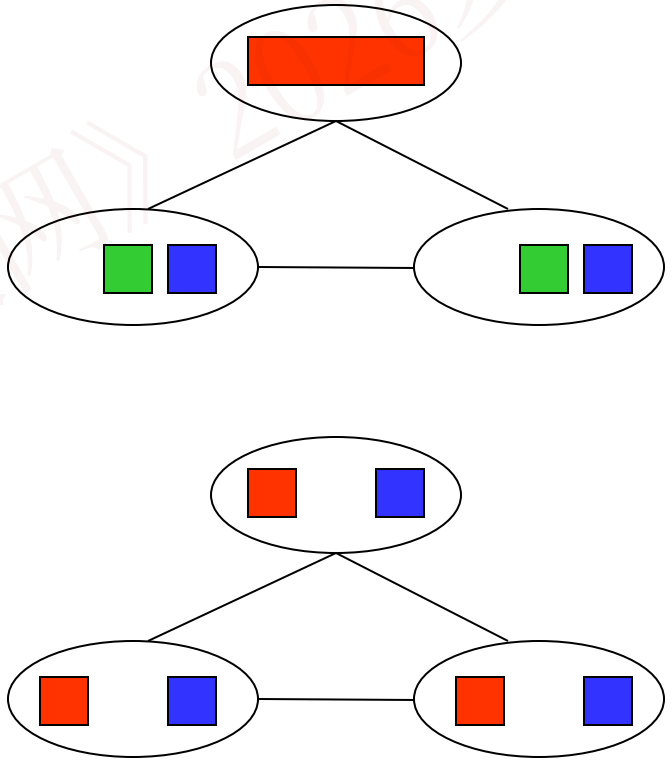
				A
B				
C				
D				
E				



- Queen 1, e.g., A & B: (1,3) (1,4) (1,5) (2,4) (2,5) (3,1) (3,5) (4,1) (4,2) (5,1) (5,2)
- Queen 2, e.g., A & C: (1,2) (1,4) (1,5) (2,1) (2,3) (2,5) (3,2) (3,4) (4,1) (4,3) (4,5) (5,1) (5,2) (5,4)
- Queen 3, e.g., A & D: (1,2) (1,3) (1,5) (2,1) (2,3) (2,4) (3,1) (3,2) (3,4) (3,5) (4,2) (4,3) (4,5) (5,1) (5,3) (5,4)
- Queen 4, e.g., A & E: (1,2) (1,3) (1,4) (2,1) (2,3) (2,4) (2,5) (3,1) (3,2) (3,4) (3,5) (4,1) (4,2) (4,3) (4,5) (5,2) (5,3) (5,4)

Limitations of Arc Consistency

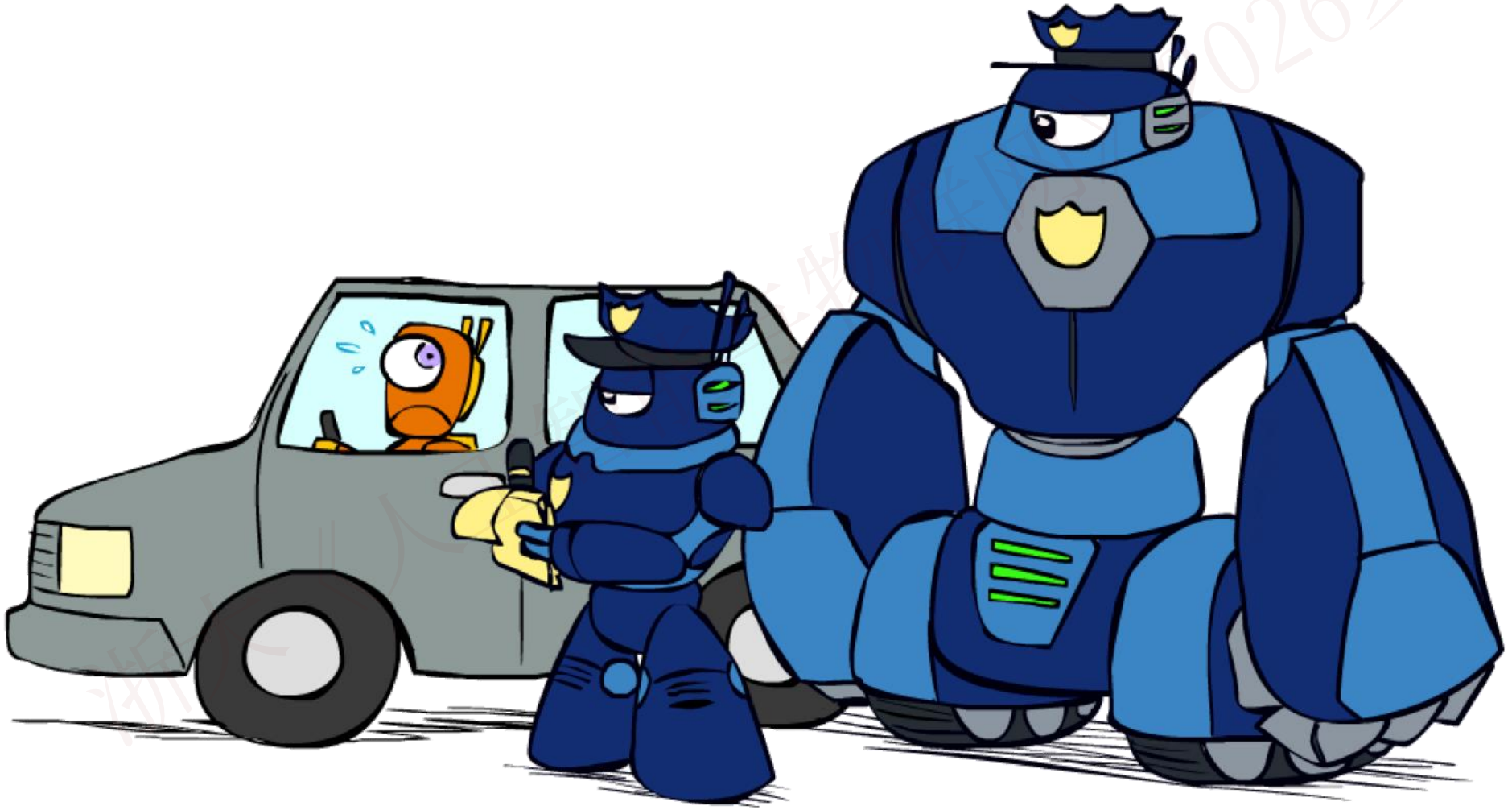
- After enforcing arc consistency:
  - Can have one solution left
  - Can have multiple solutions left
  - Can have no solutions left (and not know it)



What went wrong here?

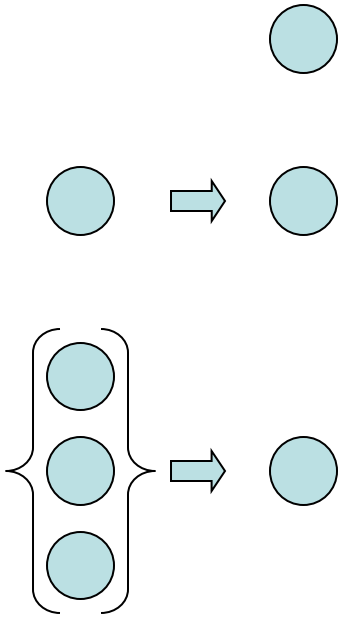
浙大《人工智能与物联网》2026夏

K-Consistency

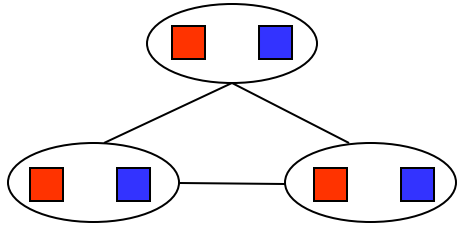


K-Consistency

- Increasing degrees of consistency
  - 1-Consistency (Node Consistency): Each single node's domain has a value which meets that node's unary constraints
  - 2-Consistency (Arc Consistency): For each pair of nodes, any consistent assignment to one can be extended to the other
  - K-Consistency: For each k nodes, any consistent assignment to k-1 can be extended to the kth node.



- Higher k more expensive to compute
- (You need to know the k=2 case: arc consistency)



浙大《人工智能导论》2026夏

Strong K-Consistency

- Strong k-consistency: also k-1, k-2, … 1 consistent
- Claim: strong n-consistency means we can solve without backtracking!
- Why?
  - Choose any assignment to any variable
  - Choose a new variable
  - By 2-consistency, there is a choice consistent with the first
  - Choose a new variable
  - By 3-consistency, there is a choice consistent with the first 2
  - …
- Lots of middle ground between arc consistency and n-consistency! (e.g. k=3, called path consistency)

浙大《人工智能与物联网》2026夏

Ordering



浙大《人工智能与物联网》2026夏

Ordering: Minimum Remaining Values

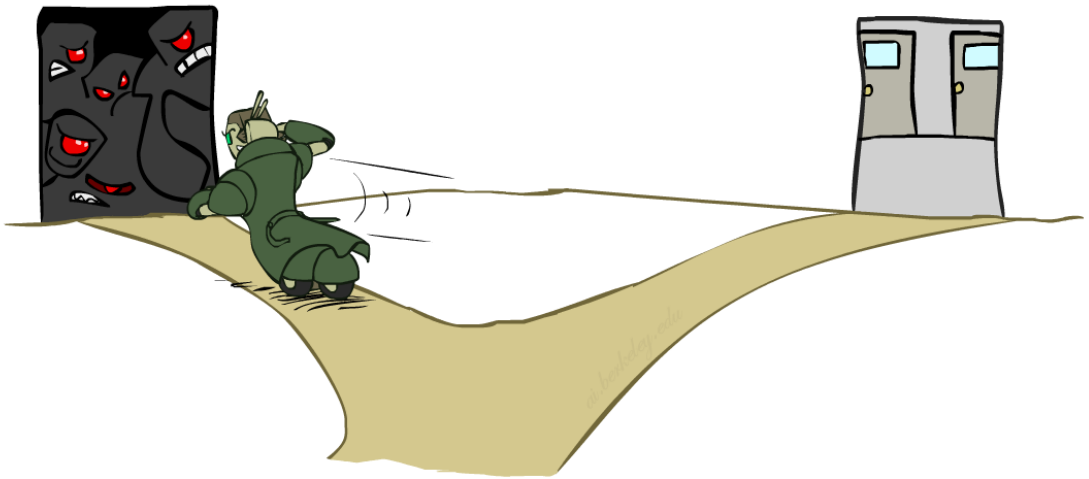
- Variable Ordering: Minimum remaining values (MRV 最少剩余值):
  - Choose the variable with the fewest *legal left values* in its domain



- Why min rather than max?
- Also called “most constrained variable

最受约束变量”

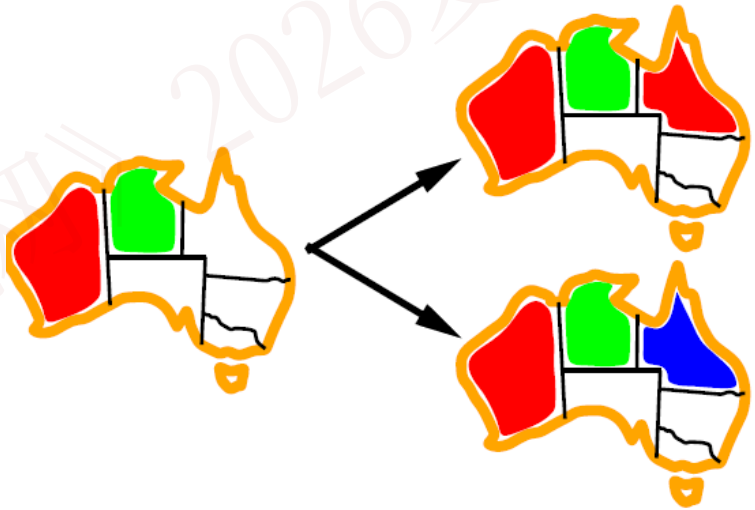
- “Fail-fast” ordering 失败优先



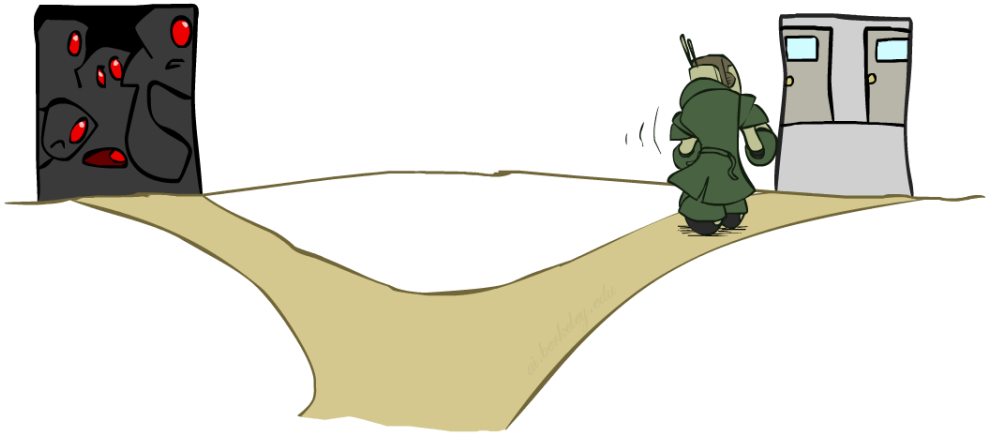
浙大《人工智能与物联网》2024夏

Ordering: Least Constraining Value

- Value Ordering: Least Constraining Value 最少约束值
  - Given a choice of variable, choose the *least constraining value*
  - I.e., the one that rules out the fewest values in the remaining variables
  - Note that it may take some computation to determine this! (E.g., rerunning filtering)

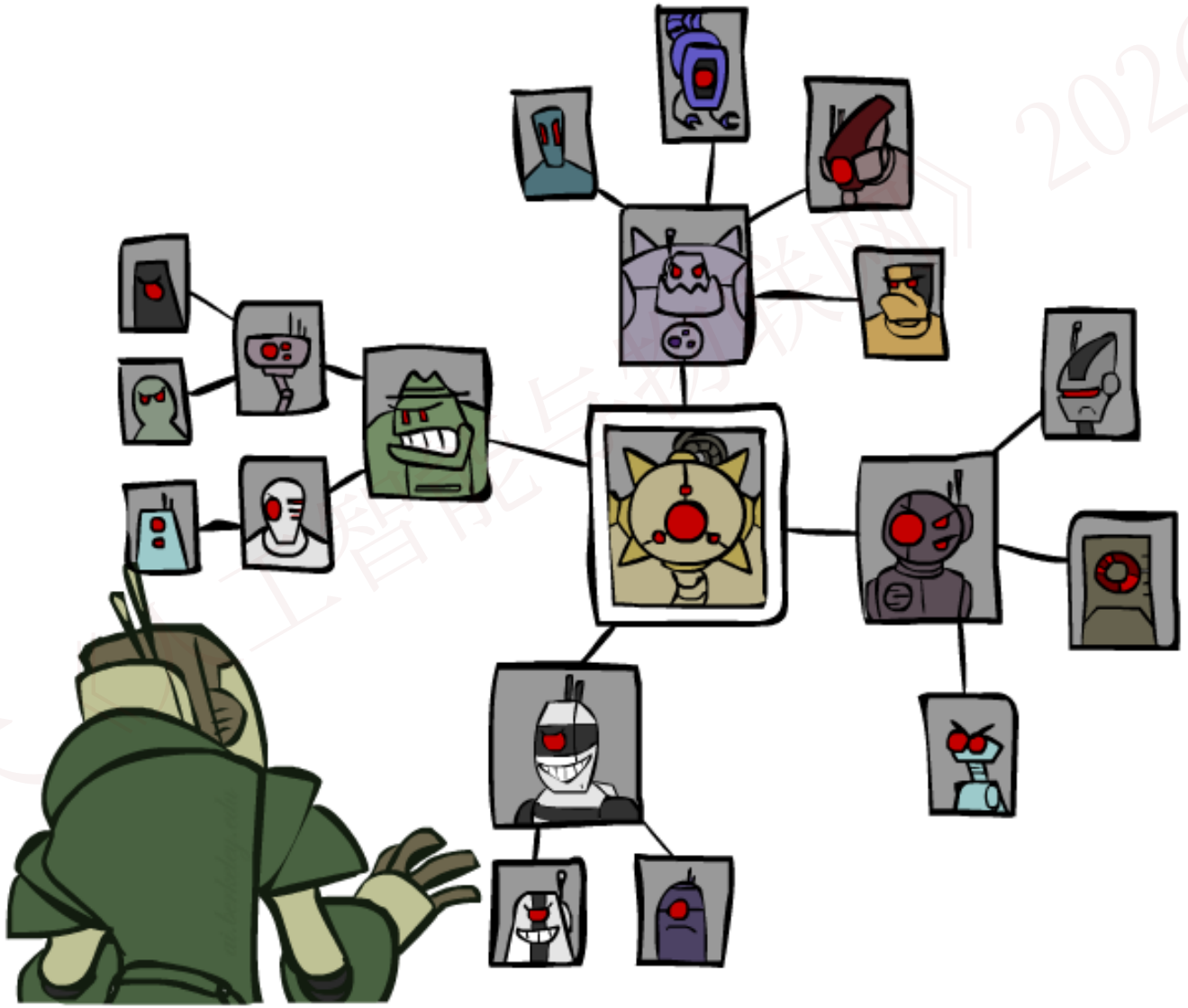


- Combining these ordering ideas makes 1000 queens feasible



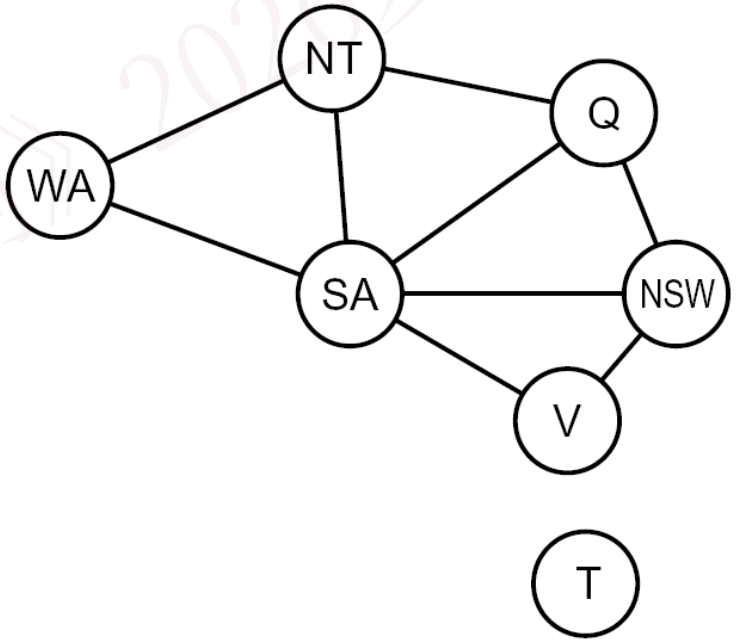
浙大《人工智能与物联网》2026夏

Structure

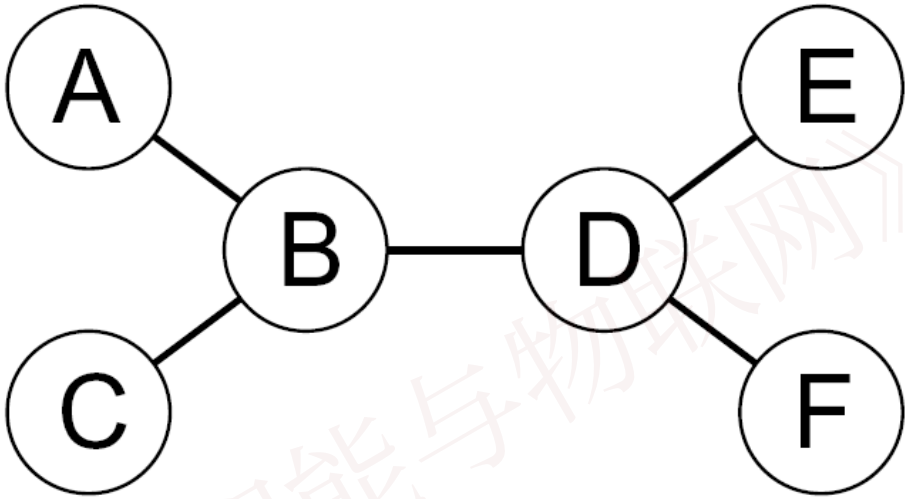


Problem Structure

- Extreme case: independent subproblems 独立子问题
  - Example: Tasmania and mainland do not interact
- Independent subproblems are identifiable as connected components of constraint graph
- Suppose a graph of n variables can be broken into subproblems of only c variables:
  - Worst-case solution cost is $O((n/c)(d^c))$, linear in n
  - E.g., n = 80, d = 2, c =20
  - 2^{80} = 4 billion years at 10 million nodes/sec
  - $(4)(2^{20})$ = 0.4 seconds at 10 million nodes/sec



Tree-Structured (树结构) CSPs

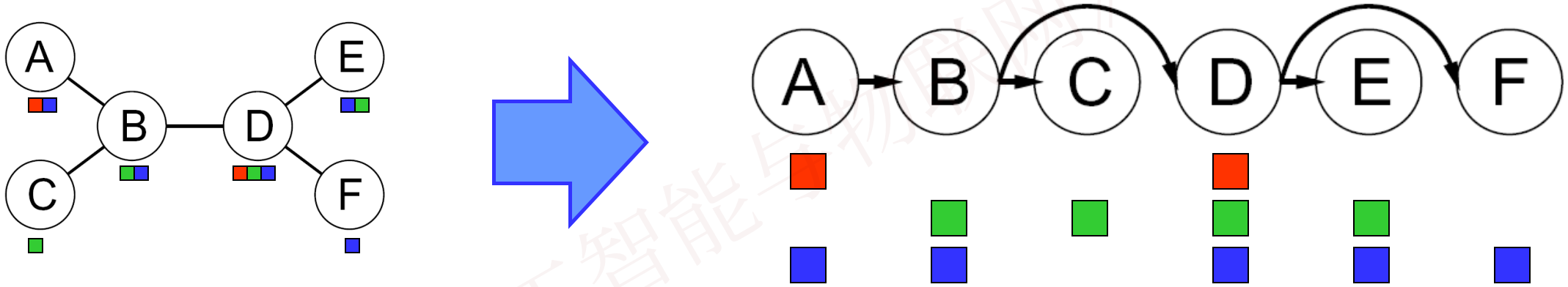


- Theorem: if the constraint graph has no loops, the CSP can be solved in $O(n\ d^2)$ time
  - Compare to general CSPs, where worst-case time is $O(d^n)$

浙大《人工智能与物联网》2026夏

Tree-Structured CSPs

- Algorithm for tree-structured CSPs:
  - Order: Choose a root variable, order variables so that parents are before children



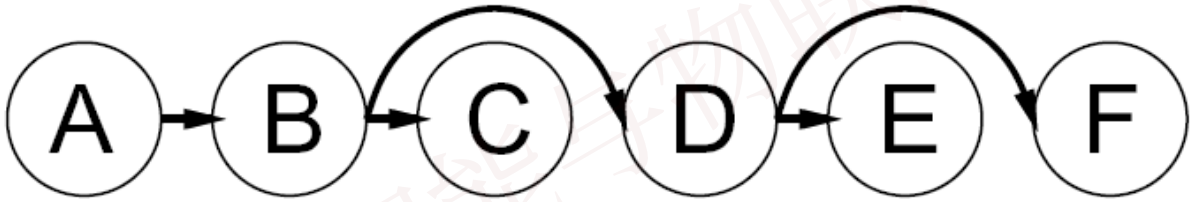
  - Remove backward: For i = n : 2, apply RemoveInconsistent(Parent(X_i),X_i)
  - Assign forward: For i = 1 : n, assign X_i consistently with Parent(X_i)

- Runtime: $O(n\ d^2)$



Tree-Structured CSPs

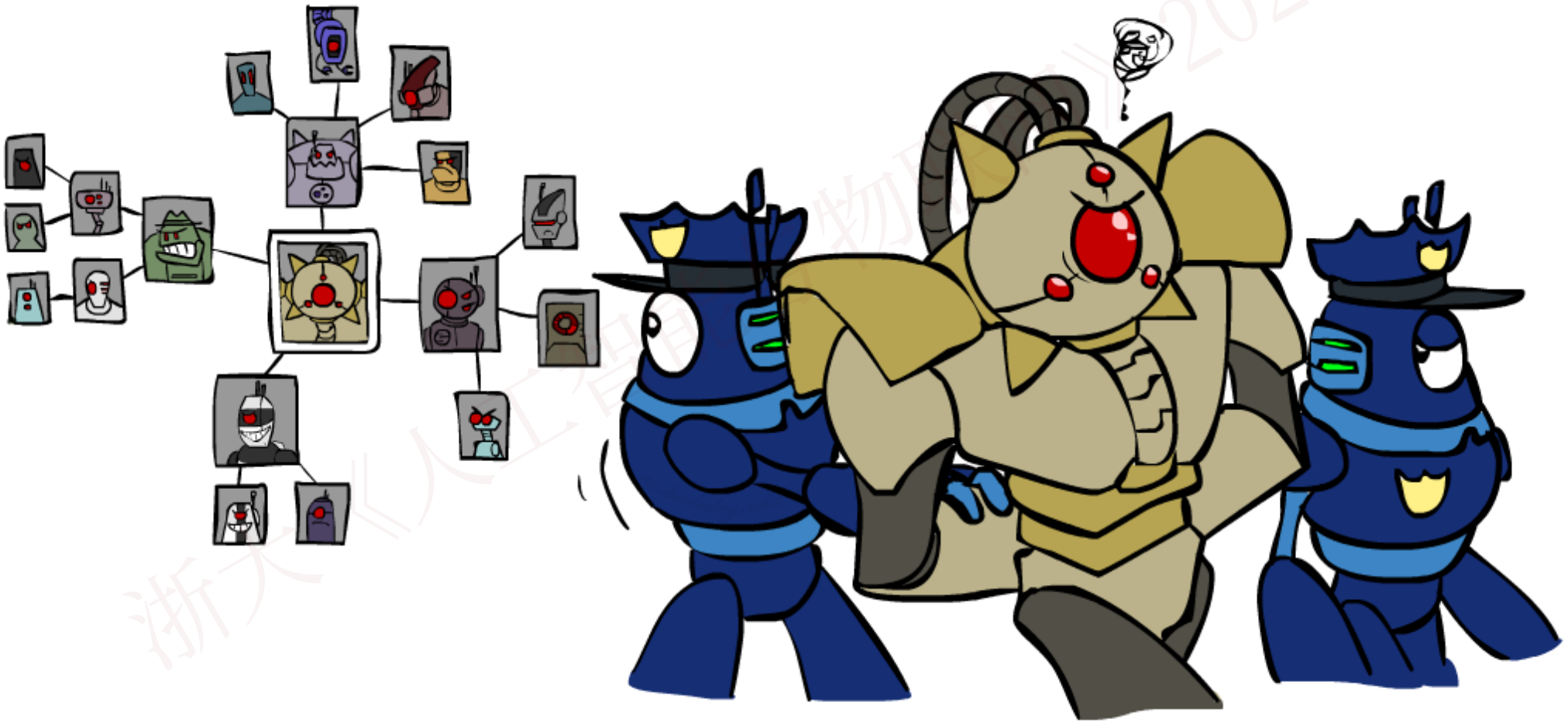
- Claim 1: After backward pass, all parent-to-child arcs are consistent
- Proof: Each X→Y was made consistent at one point and Y's domain could not have been reduced thereafter (because Y's children were processed before Y)



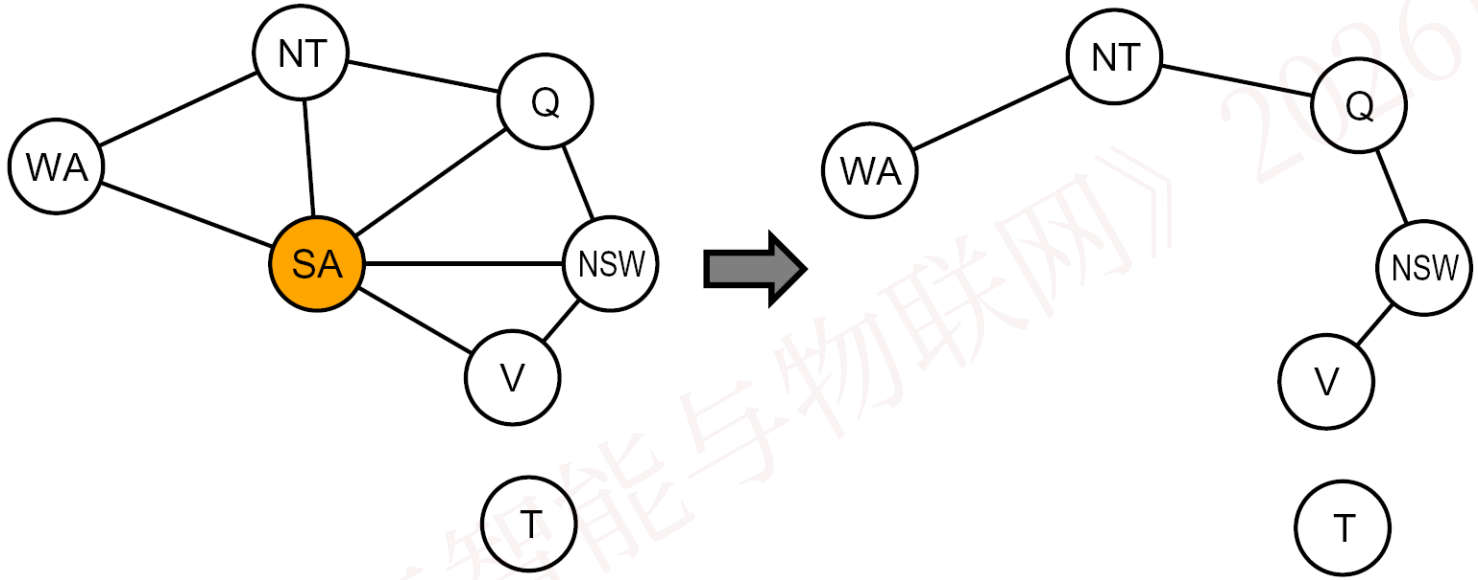
- Claim 2: If parent-to-child arcs are consistent, forward assignment will not backtrack
- Proof: Induction on position

- Why doesn't this algorithm work with cycles in the constraint graph?

Improving Structure



Nearly Tree-Structured CSPs



- Conditioning: assign a variable, prune its neighbors' domains
- Cutset conditioning (割集调整) : assign (in all ways) a set of variables such that the remaining constraint graph is a tree
- Cutset size c gives runtime O((d^c) (n-c) d^2), very fast for small c

湖大《人工智能与物联网》2026夏

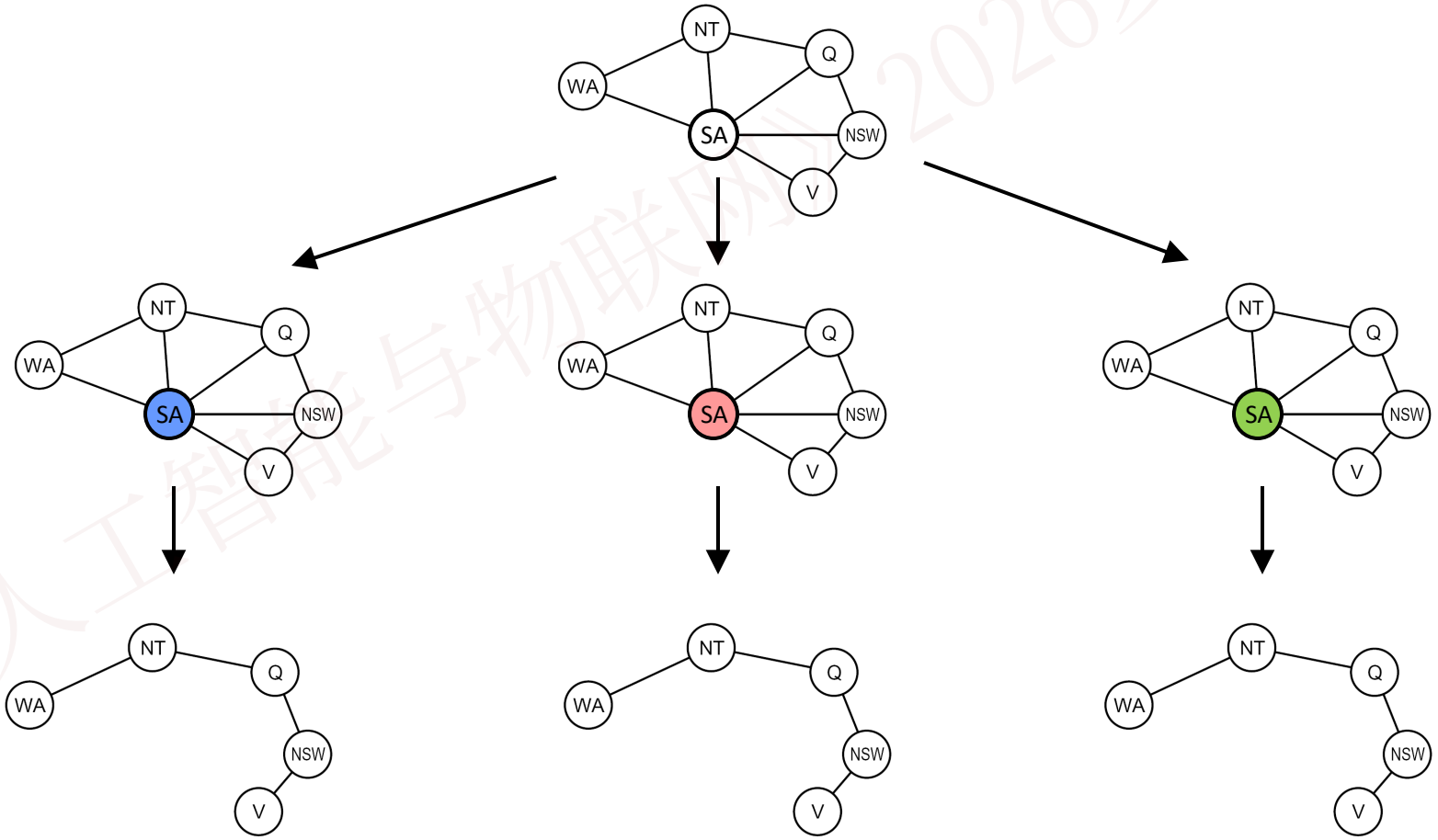
Cutset Conditioning

Choose a cutset

Assign the cutset (all possible ways)

Compute residual CSP for each assignment

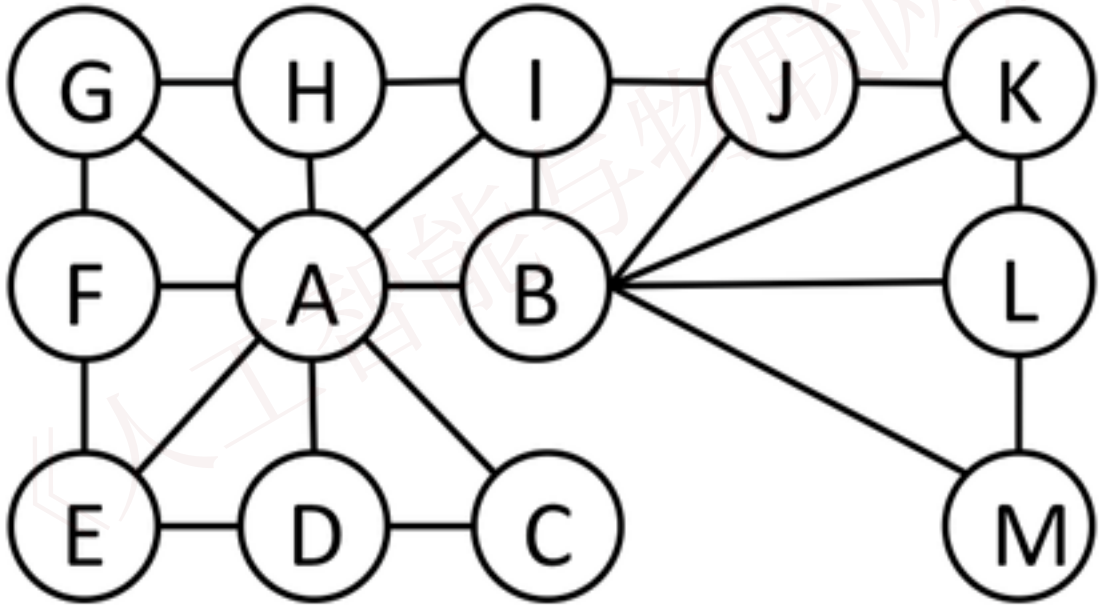
Solve the residual CSPs (tree structured)



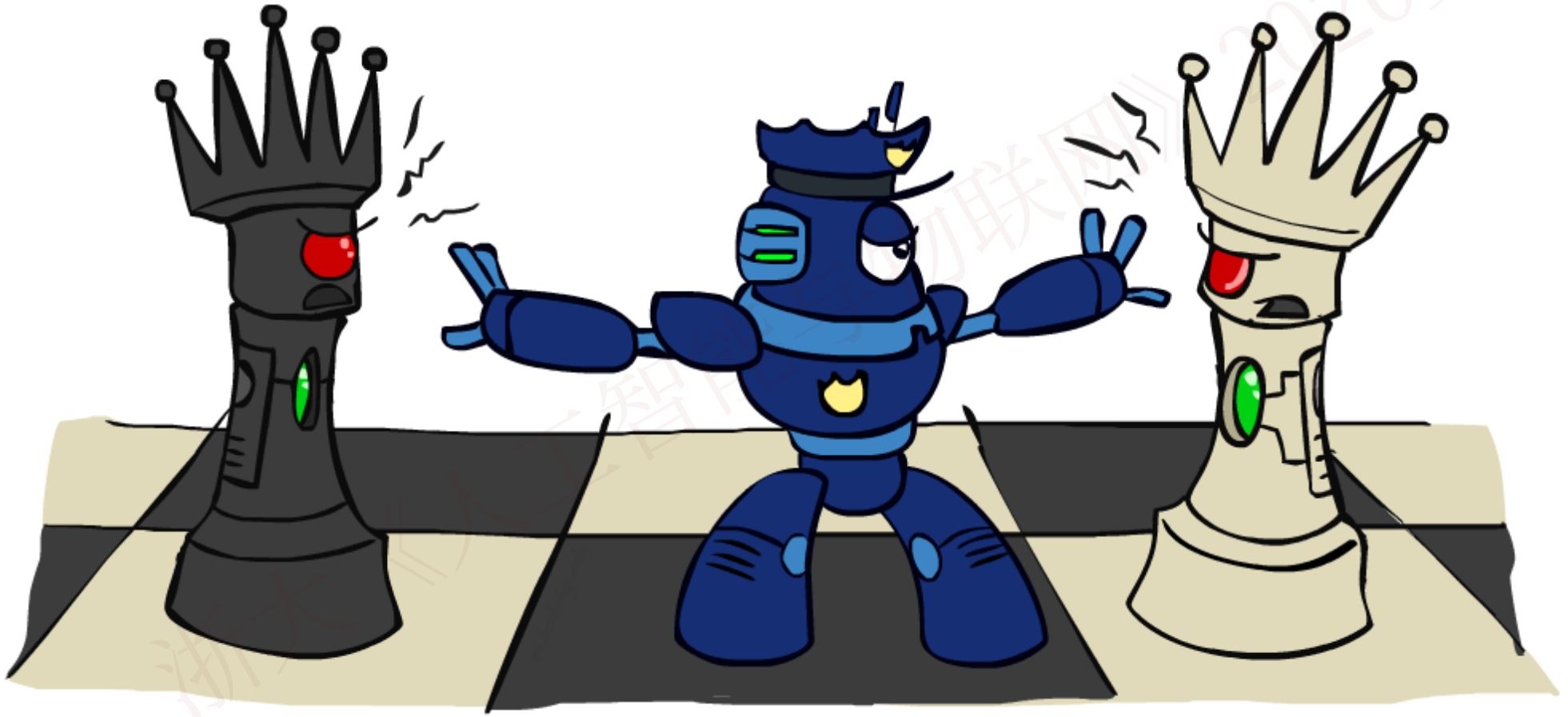
《人工智能与物联网》2026夏

Cutset Quiz

- Find the smallest cutset for the graph below.



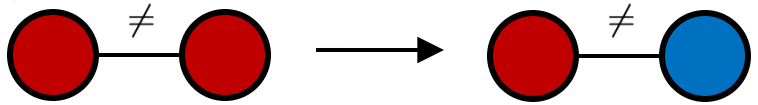
Iterative Improvement (迭代优化)



Iterative Algorithms for CSPs

- Local search (局部搜索) methods typically work with "complete" states, i.e., all variables assigned

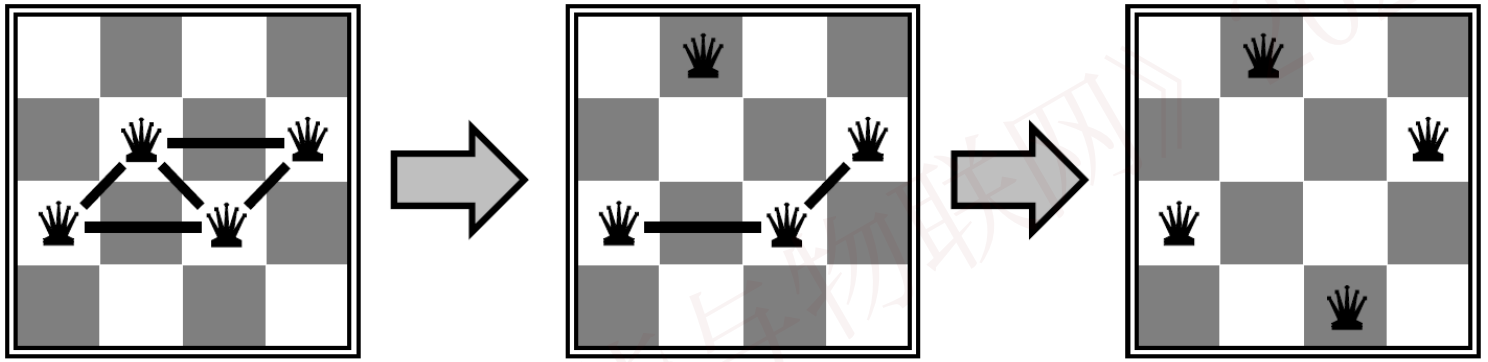
- To apply to CSPs:
  - Take an assignment with unsatisfied constraints
  - *Reassign* variable values
  - No fringe! Live on the edge.



- Algorithm: While not solved,
  - Variable selection: randomly select any conflicted variable
  - Value selection: min-conflicts heuristic 最少冲突启发式:
    - Choose a value that violates the fewest constraints
    - I.e., hill climb with h(n) = total number of violated constraints

西交《人工智能与物联网》2025夏

Example: 4-Queens



- States: 4 queens in 4 columns ($4^4 = 256$ states)
- Actions: move queen in column
- Goal test: no attacks
- Evaluation: h(n) = number of attacks

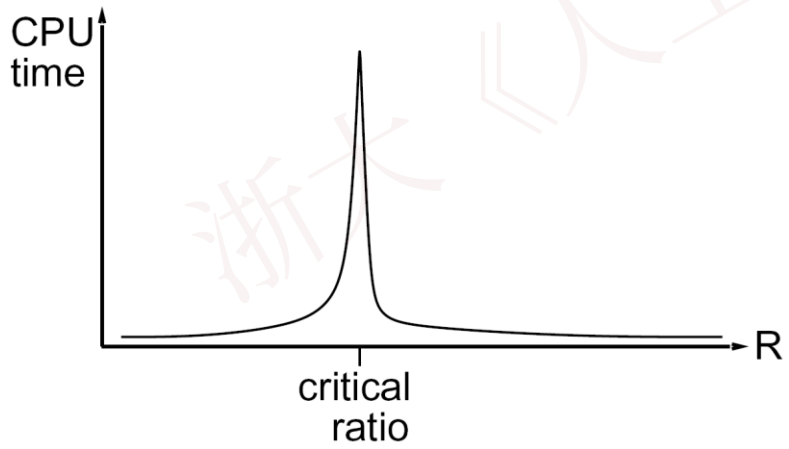
浙大人工智能与机器人研究所 2026夏

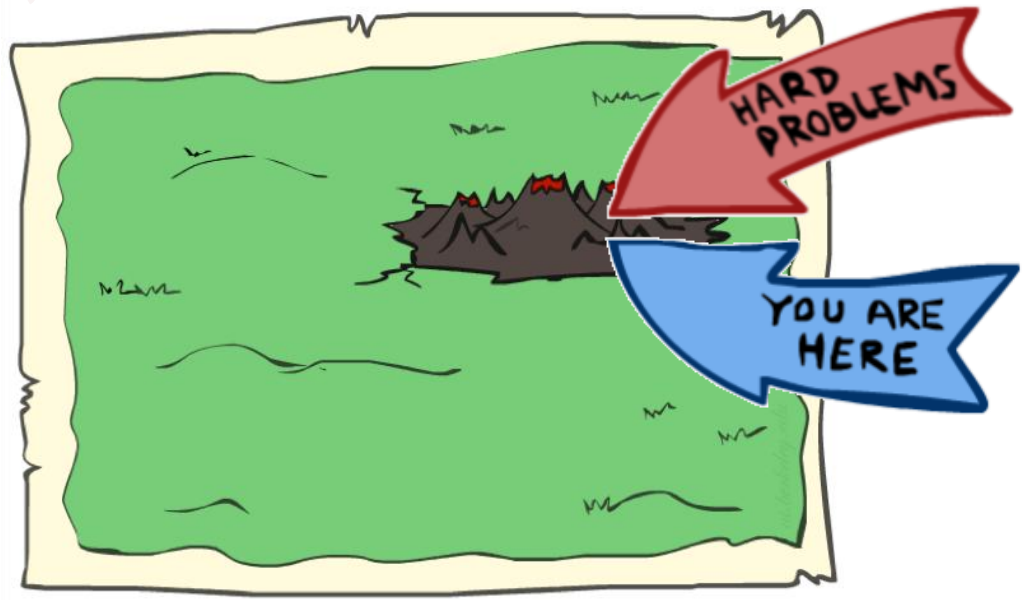
Performance of Min-Conflicts

- Given random initial state, can solve n-queens in almost constant time for arbitrary n with high probability (e.g., n = 10,000,000)!

- The same appears to be true for any randomly-generated CSP *except* in a narrow range of the ratio

$$R = \frac{\text{number of constraints}}{\text{number of variables}}$$

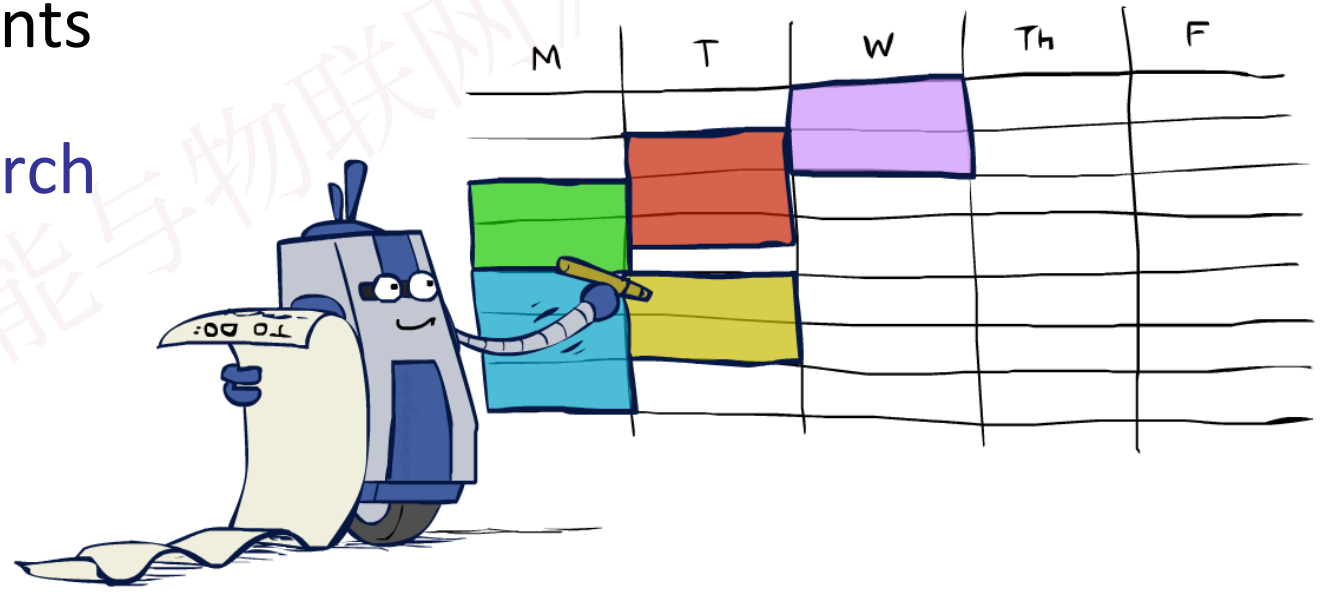




浙大《人工智能与物联网》2026春

Summary: CSPs

- CSPs are a special kind of search problem:
  - States are partial assignments
  - Goal test defined by constraints
- Basic solution: backtracking search
- Speed-ups:
  - Ordering
  - Filtering
  - Structure
- Iterative min-conflicts is often effective in practice



浙大《人工智能与物联网》2026夏

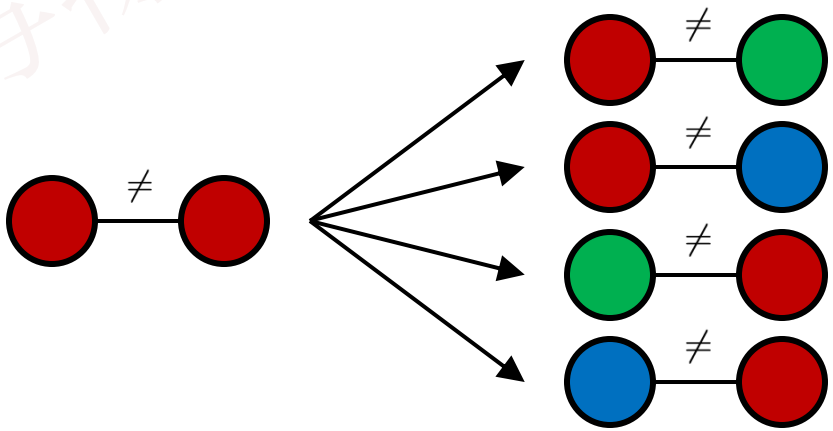
Local Search



浙大人工智能与物联网 2026夏

Local Search

- Tree search keeps unexplored alternatives on the fringe (ensures completeness)
- Local search: improve a single option until you can't make it better (no fringe!)
- New successor function: local changes

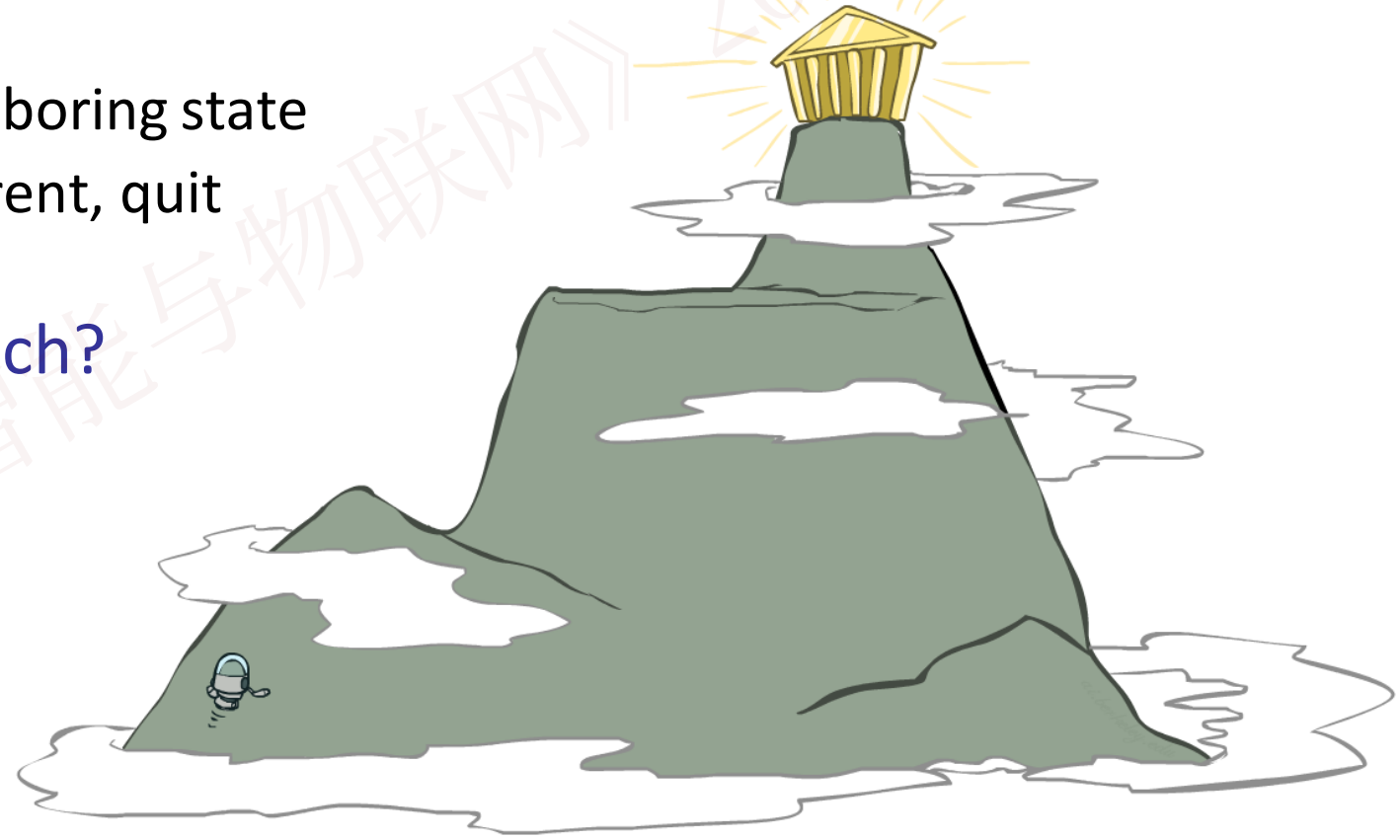


- Generally much faster and more memory efficient (but incomplete and suboptimal)

浙大《人工智能与物联网》2025夏

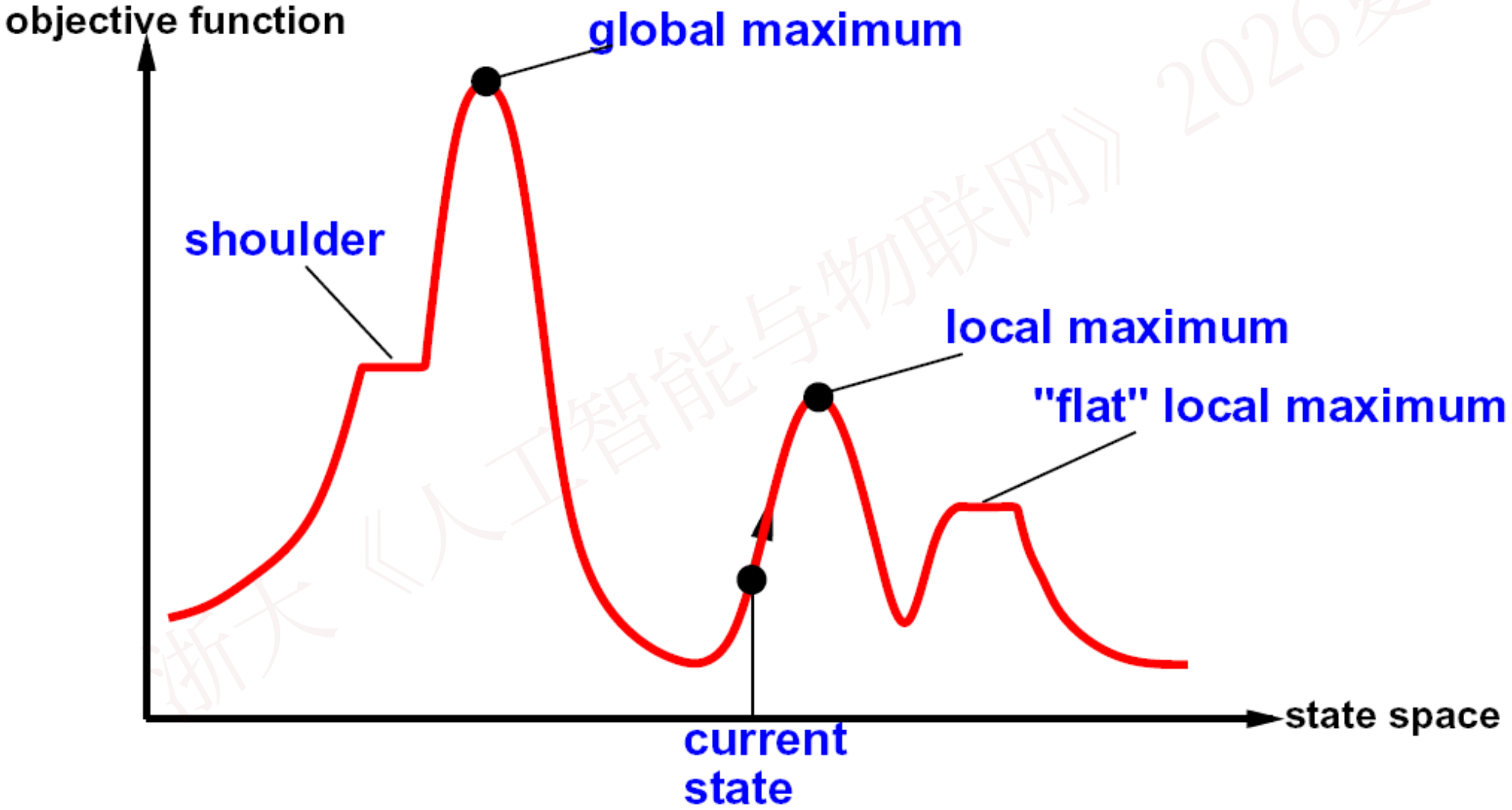
Hill Climbing (爬山法)

- Simple, general idea:
  - Start wherever
  - Repeat: move to the best neighboring state
  - If no neighbors better than current, quit
- What's bad about this approach?
  - Complete?
  - Optimal?
- What's good about it?



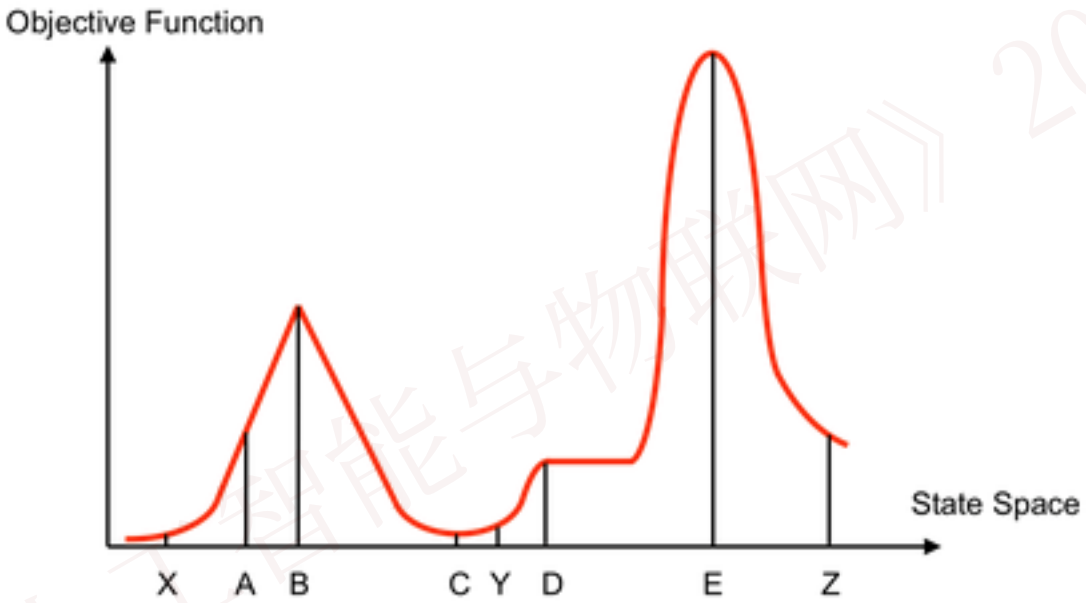
浙大《人工智能与物联网》2026夏

Hill Climbing Diagram



浙大《人工智能与物联网》2026夏

Hill Climbing Quiz



Starting from X, where do you end up ?

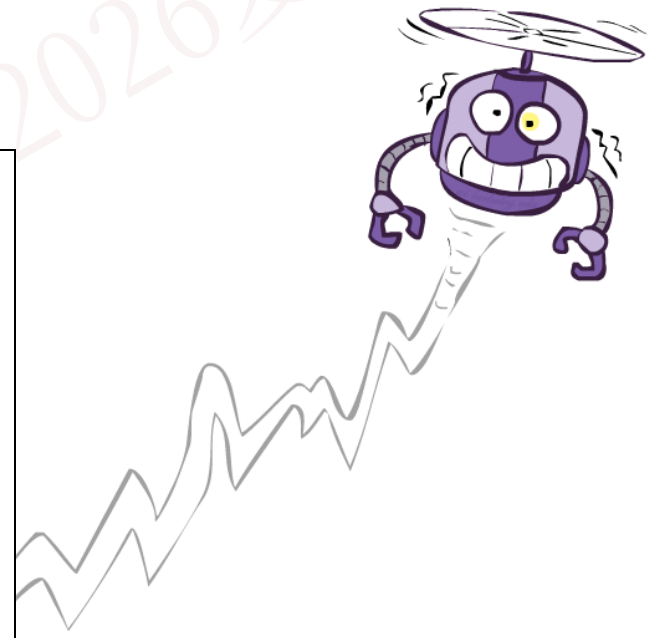
Starting from Y, where do you end up ?

Starting from Z, where do you end up ?

浙大《人工智能与物联网》2026夏

Simulated Annealing (模拟退火算法)

- Idea: Escape local maxima by allowing downhill moves
  - But make them rarer as time goes on

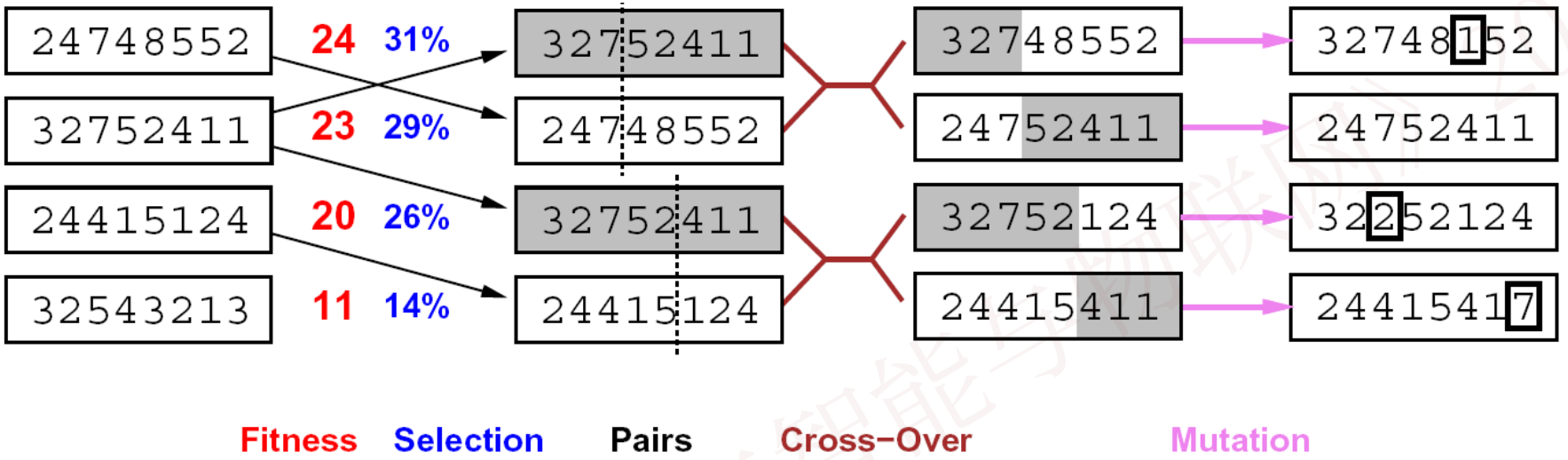


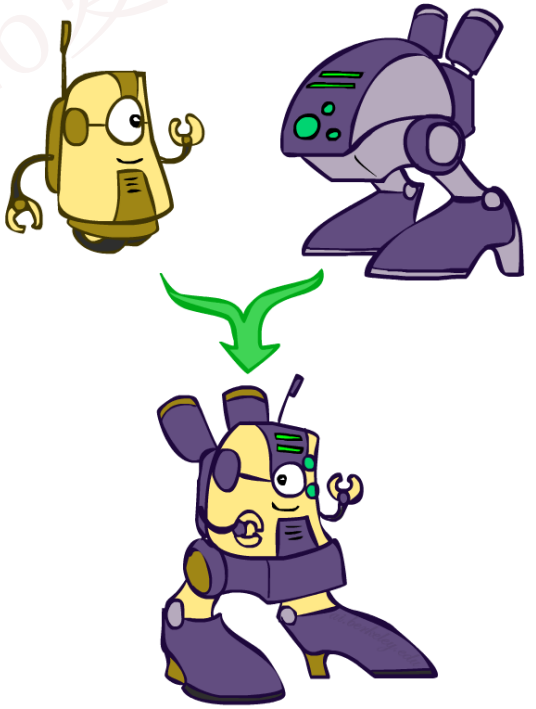
```
function SIMULATED-ANNEALING(problem, schedule) returns a solution state
  inputs: problem, a problem
          schedule, a mapping from time to "temperature"
  local variables: current, a node
                   next, a node
                   T, a "temperature" controlling prob. of downward steps

  current ← MAKE-NODE(INITIAL-STATE[problem])
  for t ← 1 to ∞ do
      T ← schedule[t]
      if T = 0 then return current
      next ← a randomly selected successor of current
      ΔE ← VALUE[next] – VALUE[current]
      if ΔE > 0 then current ← next
      else current ← next only with probability e^(Δ E/T)
```

浙大《人工智能与物联网》2026夏

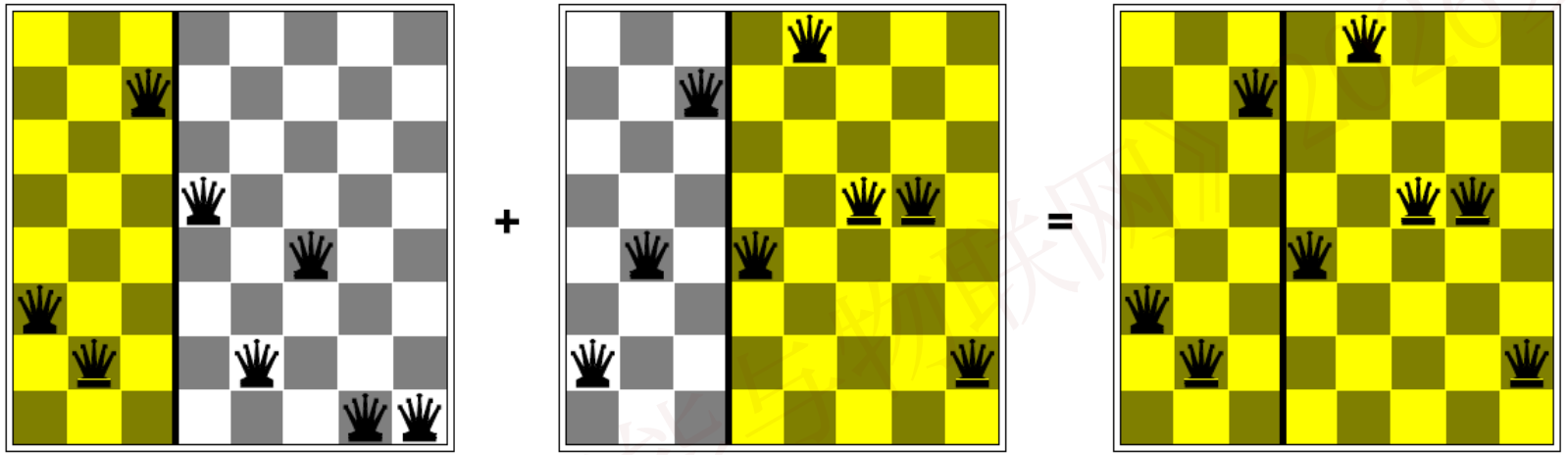
Genetic Algorithms (遗传算法)





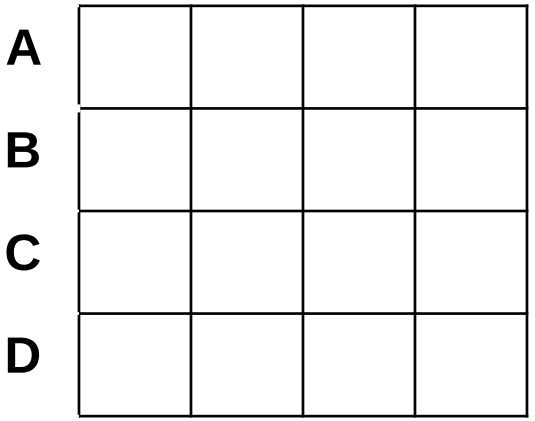
- Genetic algorithms imitate a natural selection (自然选择)
  - Keep best N candidates at each step (selection) based on a fitness function (适应函数)
  - Also have pairwise crossover (交叉), with optional mutation (变异) to give variety

Example: N-Queens



- Why does crossover make sense here?
- When wouldn't it make sense?
- What would mutation be?
- What would a good fitness function be?

Enforcing Arc Consistency: 4-Queens

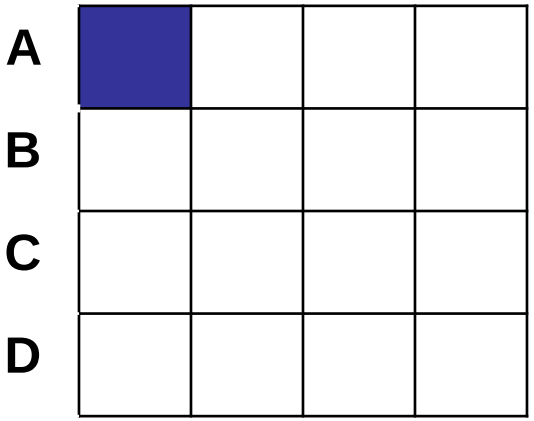


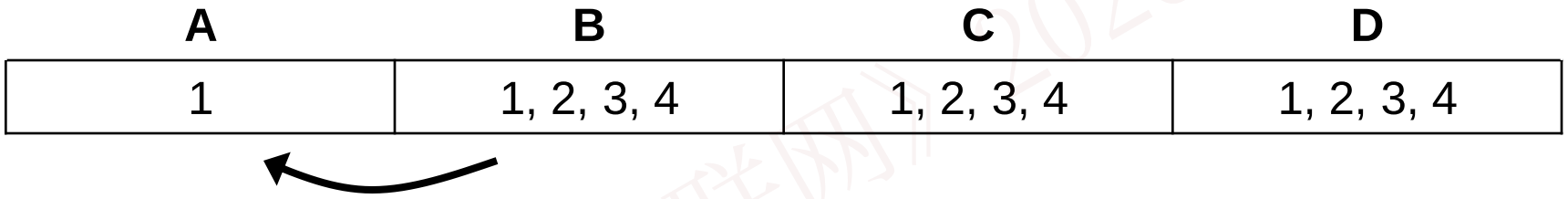
A	B	C	D
1, 2, 3, 4	1, 2, 3, 4	1, 2, 3, 4	1, 2, 3, 4

queue **A→B, A→C, A→D, B→A, B→C, B→D, C→A, C→B, C→D, D→A, D→B, D→C**

浙大《人工智能与物联网》2026夏

Enforcing Arc Consistency: 4-Queens



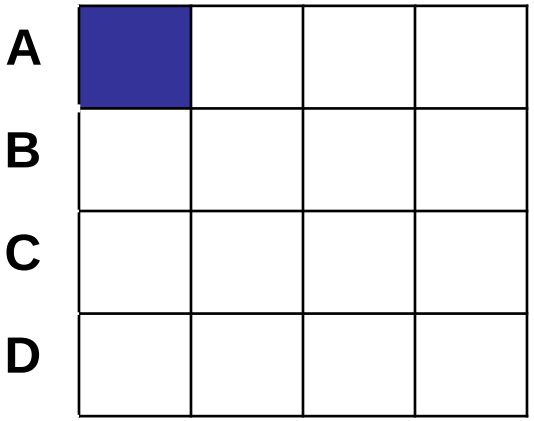


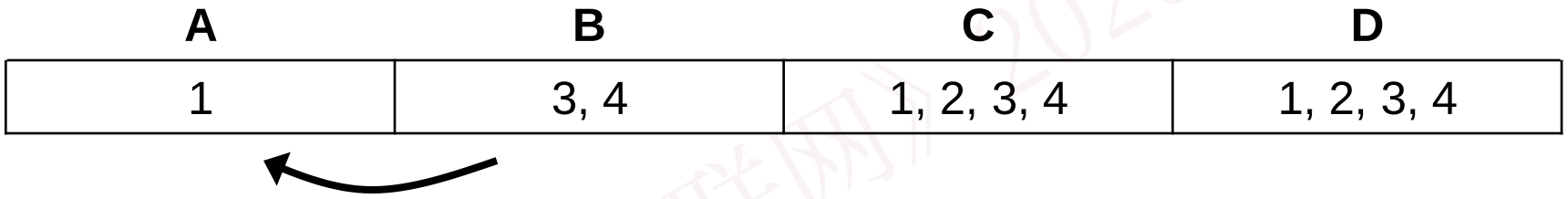
All arc A→B, A→C, A→D, B→A, B→C, B→D, C→A, C→B, C→D, D→A, D→B, D→C

queue B→A, B→C, B→D, C→A, C→B, C→D, D→A, D→B, D→C

浙大人工智能与物联网 2026夏

Enforcing Arc Consistency: 4-Queens



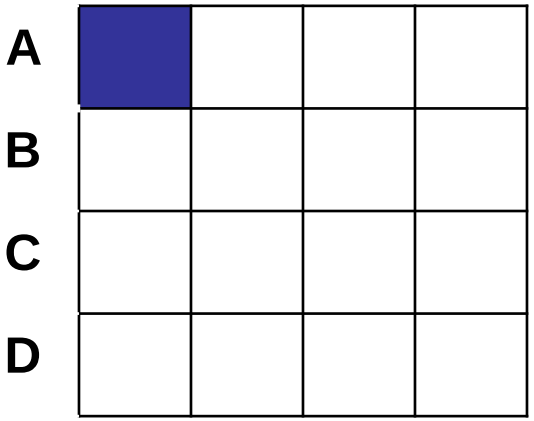


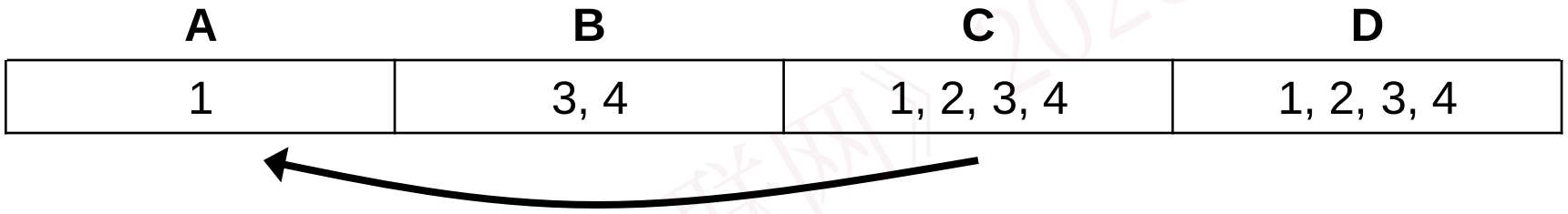
All arc A→B, A→C, A→D, B→A, B→C, B→D, C→A, C→B, C→D, D→A, D→B, D→C

queue B→A, B→C, B→D, C→A, C→B, C→D, D→A, D→B, D→C, A→B

浙大人工智能与物联网 2026夏

Enforcing Arc Consistency: 4-Queens



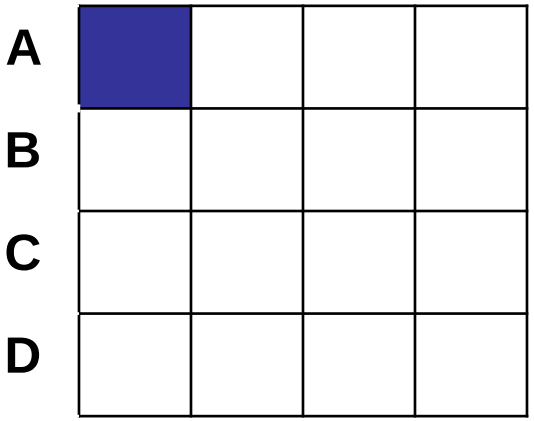


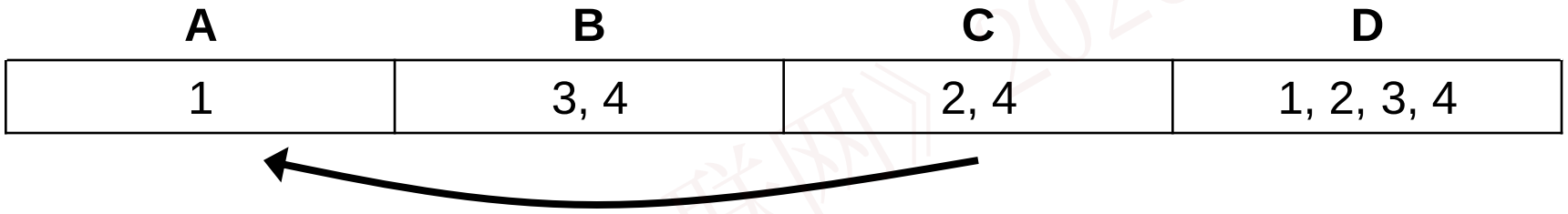
All arc A→B, A→C, A→D, B→A, B→C, B→D, C→A, C→B, C→D, D→A, D→B, D→C

queue C→A, C→B, C→D, D→A, D→B, D→C, A→B

浙大《人工智能与物联网》2026夏

Enforcing Arc Consistency: 4-Queens



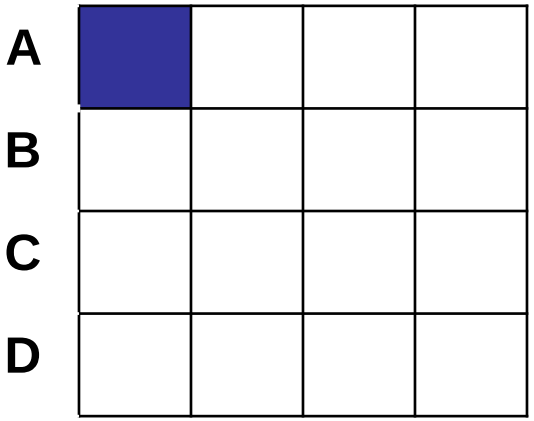


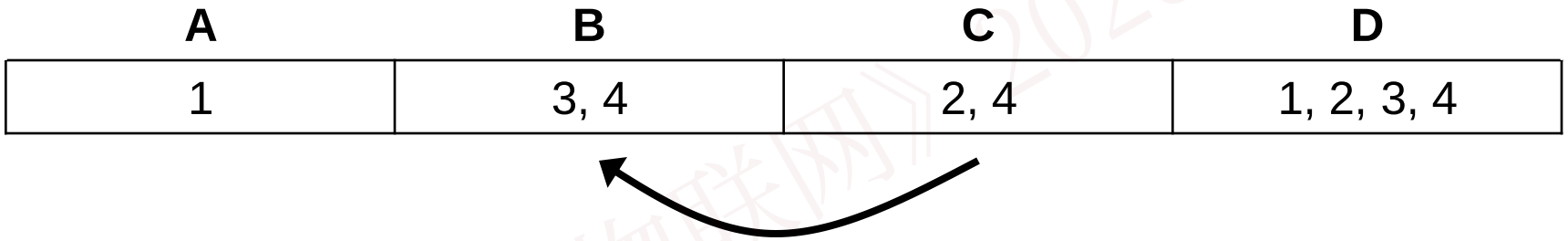
All arc A⟶B, A⟶C, A⟶D, B⟶A, B⟶C, B⟶D, C⟶A, C⟶B, C⟶D, D⟶A, D⟶B, D⟶C

queue C⟶A, C⟶B, C⟶D, D⟶A, D⟶B, D⟶C, A⟶B, A⟶C, B⟶C

浙大《人工智能与物联网》2026夏

Enforcing Arc Consistency: 4-Queens



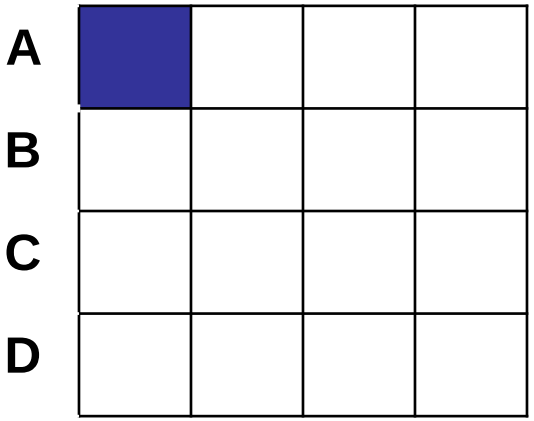


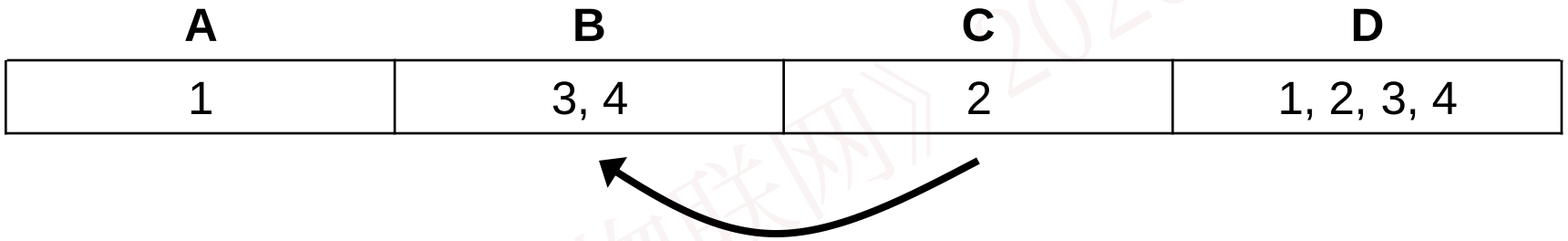
All arc A→B, A→C, A→D, B→A, B→C, B→D, C→A, C→B, C→D, D→A, D→B, D→C

queue C→B, C→D, D→A, D→B, D→C, A→B, A→C, B→C

浙大《人工智能与物联网》2026夏

Enforcing Arc Consistency: 4-Queens



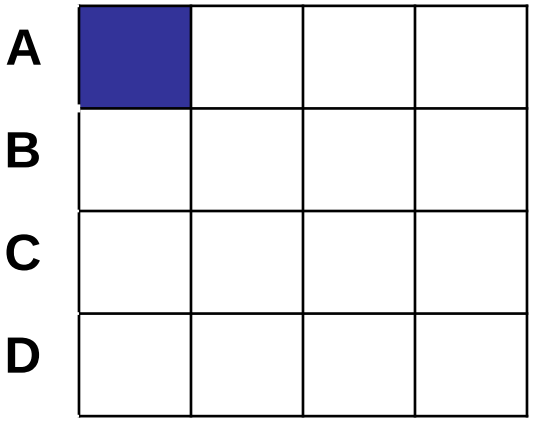


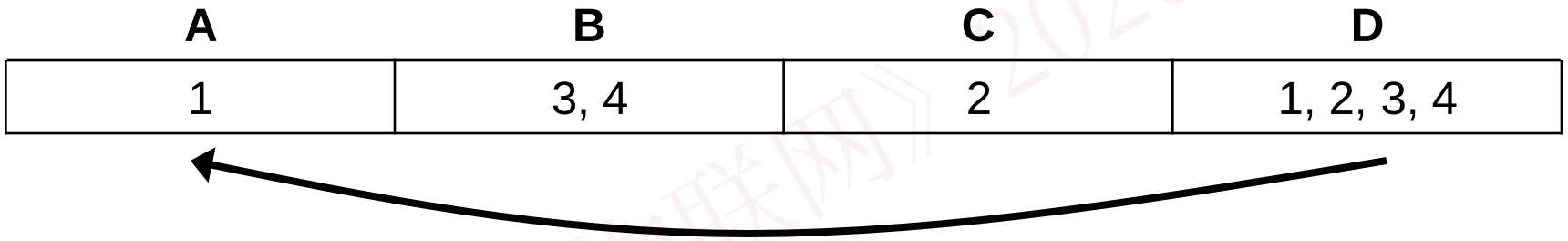
All arc A→B, A→C, A→D, B→A, B→C, B→D, C→A, C→B, C→D, D→A, D→B, D→C

queue C→B, C→D, D→A, D→B, D→C, A→B, A→C, B→C

浙大人工智能与物联网公2026夏

Enforcing Arc Consistency: 4-Queens



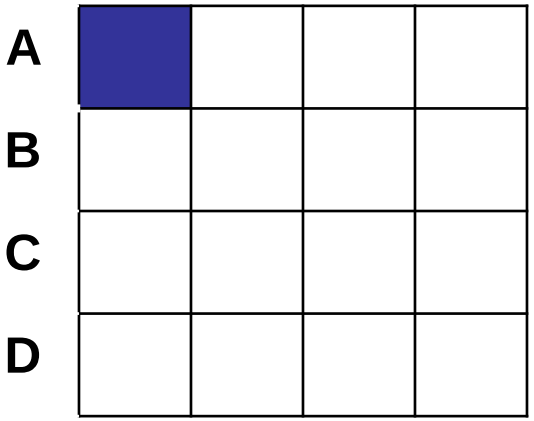


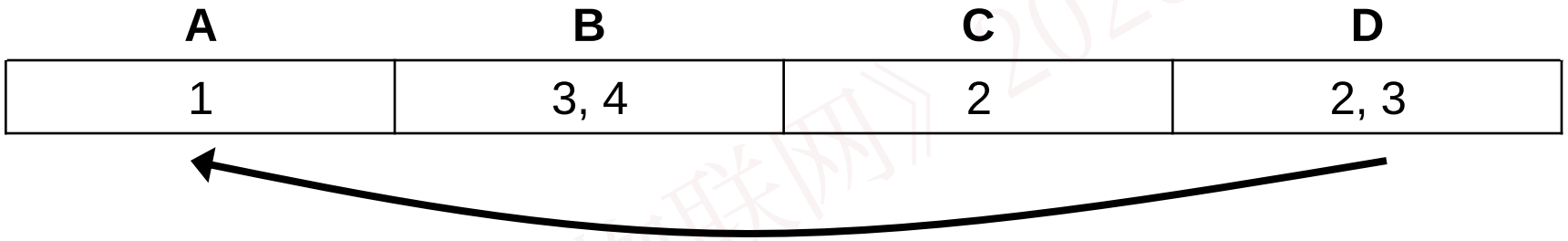
All arc A⟶B, A⟶C, A⟶D, B⟶A, B⟶C, B⟶D, C⟶A, C⟶B, C⟶D, D⟶A, D⟶B, D⟶C

queue D⟶A, D⟶B, D⟶C, A⟶B, A⟶C, B⟶C

浙大《人工智能与物联网》2026夏

Enforcing Arc Consistency: 4-Queens



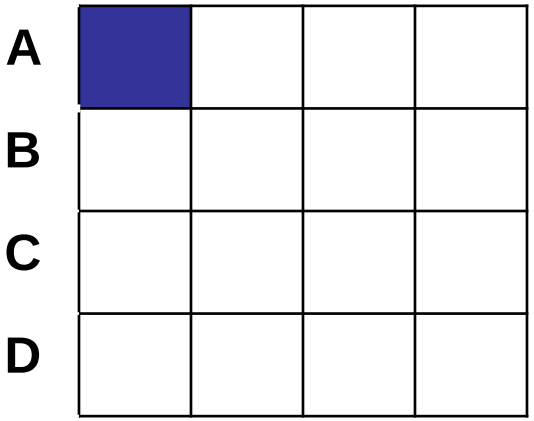


All arc A⟶B, A⟶C, A⟶D, B⟶A, B⟶C, B⟶D, C⟶A, C⟶B, C⟶D, D⟶A, D⟶B, D⟶C

queue D⟶A, D⟶B, D⟶C, A⟶B, A⟶C, B⟶C, A⟶D, B⟶D, C⟶D

浙大《人工智能与物联网》2026夏

Enforcing Arc Consistency: 4-Queens



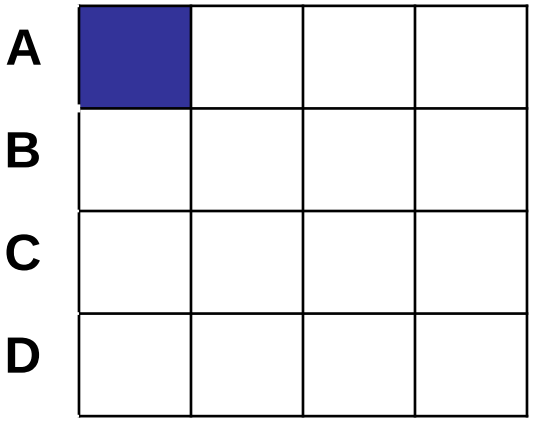


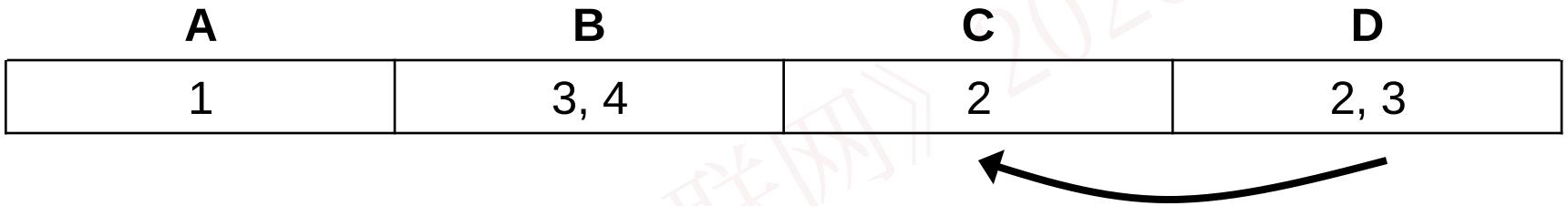
All arc A⟶B, A⟶C, A⟶D, B⟶A, B⟶C, B⟶D, C⟶A, C⟶B, C⟶D, D⟶A, D⟶B, D⟶C

queue D⟶B, D⟶C, A⟶B, A⟶C, B⟶C, A⟶D, B⟶D, C⟶D

浙大《人工智能与物联网》2026夏

Enforcing Arc Consistency: 4-Queens



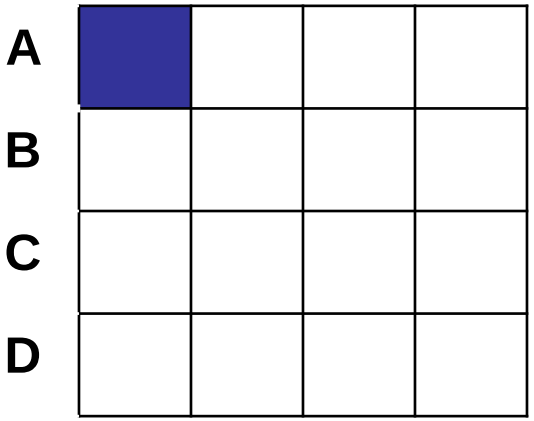


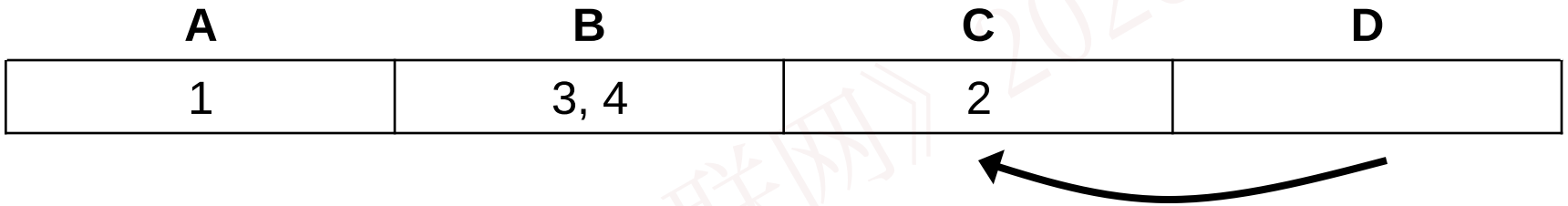
All arc A⟶B, A⟶C, A⟶D, B⟶A, B⟶C, B⟶D, C⟶A, C⟶B, C⟶D, D⟶A, D⟶B, D⟶C

queue D⟶C, A⟶B, A⟶C, B⟶C, A⟶D, B⟶D, C⟶D

浙大《人工智能与物联网》2026夏

Enforcing Arc Consistency: 4-Queens





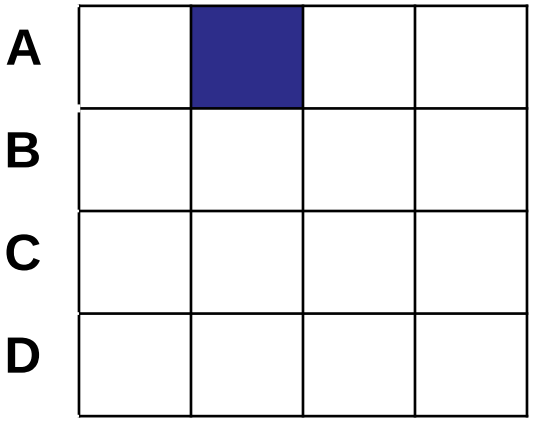
All arc A⟶B, A⟶C, A⟶D, B⟶A, B⟶C, B⟶D, C⟶A, C⟶B, C⟶D, D⟶A, D⟶B, D⟶C

queue D⟶C, A⟶B, A⟶C, B⟶C, A⟶D, B⟶D, C⟶D



浙大《人工智能与物联网》2026夏

Enforcing Arc Consistency: 4-Queens

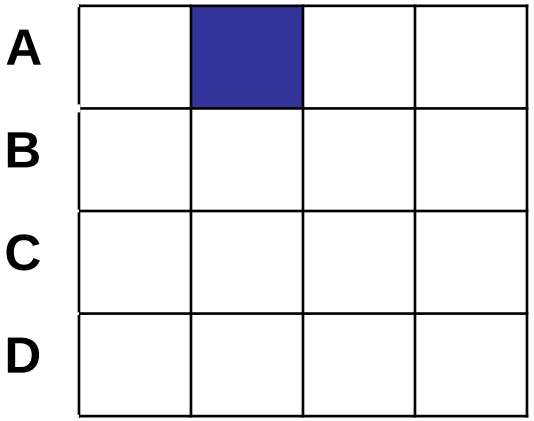


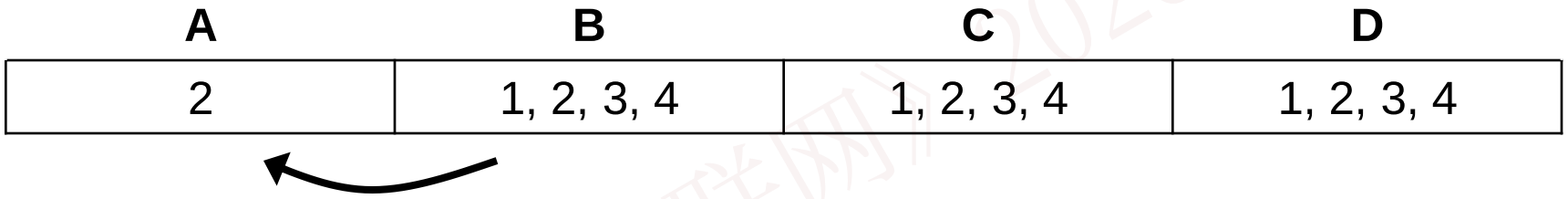
A	B	C	D
2	1, 2, 3, 4	1, 2, 3, 4	1, 2, 3, 4

queue **A→B, A→C, A→D, B→A, B→C, B→D, C→A, C→B, C→D, D→A, D→B, D→C**

浙大《人工智能与物联网》2026夏

Enforcing Arc Consistency: 4-Queens



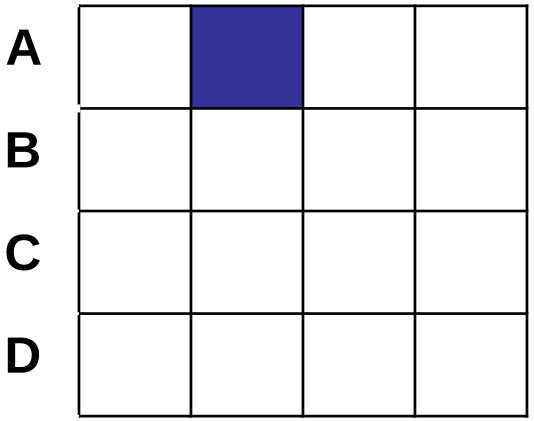


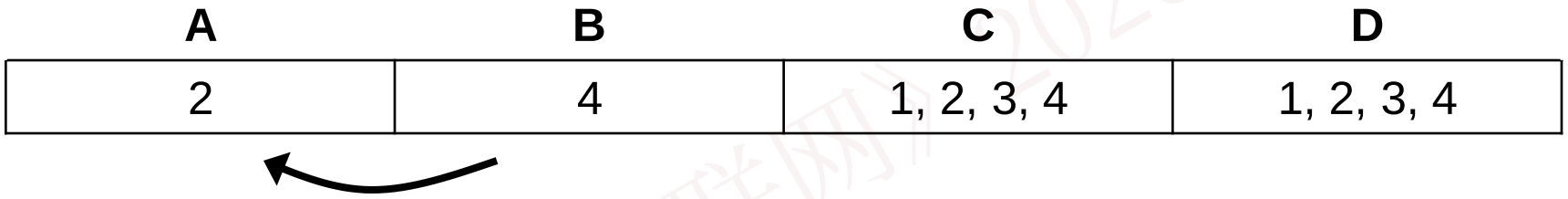
All arc A→B, A→C, A→D, B→A, B→C, B→D, C→A, C→B, C→D, D→A, D→B, D→C

queue B→A, B→C, B→D, C→A, C→B, C→D, D→A, D→B, D→C

浙大人工智能与物联网 2026夏

Enforcing Arc Consistency: 4-Queens



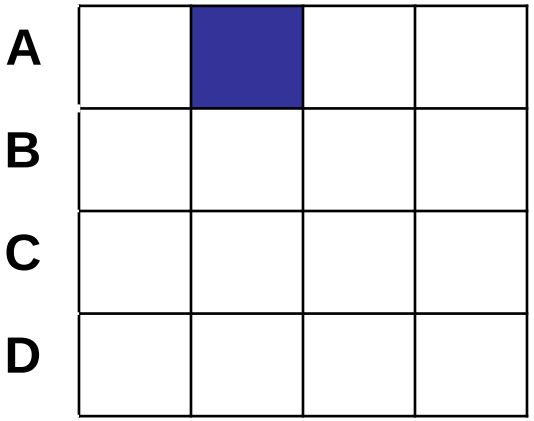


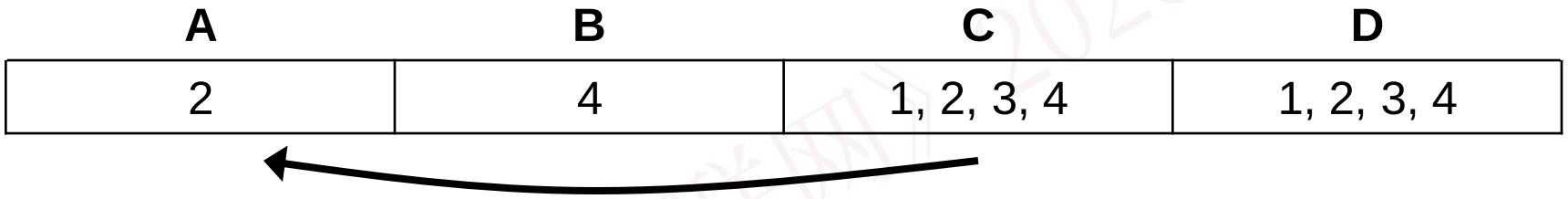
All arc A→B, A→C, A→D, B→A, B→C, B→D, C→A, C→B, C→D, D→A, D→B, D→C

queue B→A, B→C, B→D, C→A, C→B, C→D, D→A, D→B, D→C, A→B

浙大人工智能与物联网 2026夏

Enforcing Arc Consistency: 4-Queens



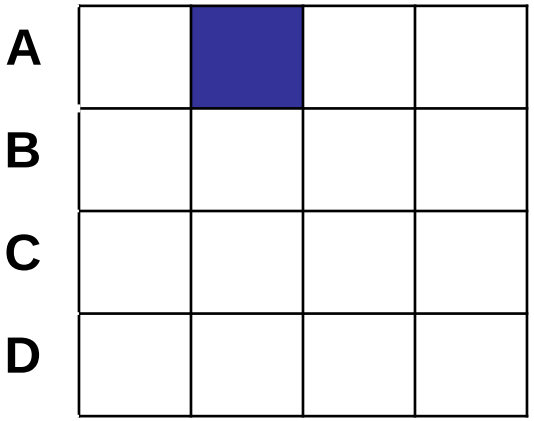


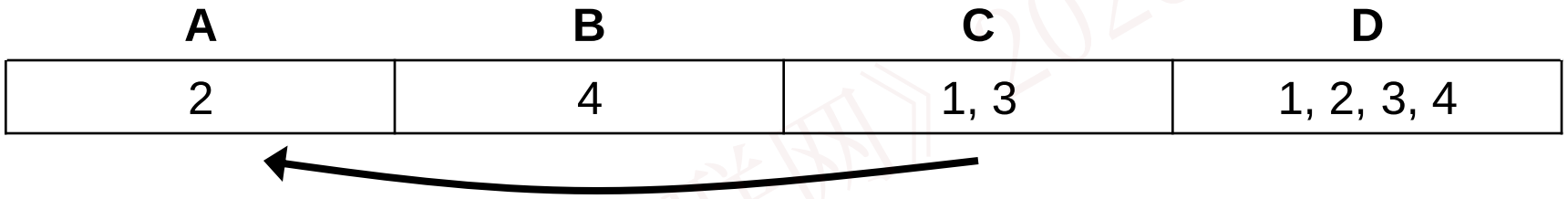
All arc A→B, A→C, A→D, B→A, B→C, B→D, C→A, C→B, C→D, D→A, D→B, D→C

queue C→A, C→B, C→D, D→A, D→B, D→C, A→B

浙大人工智能与物联网 2026夏

Enforcing Arc Consistency: 4-Queens



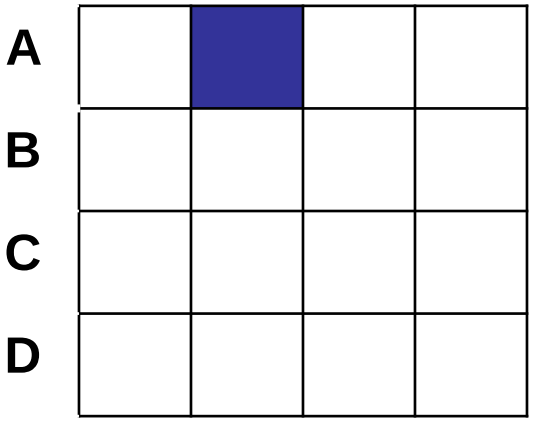


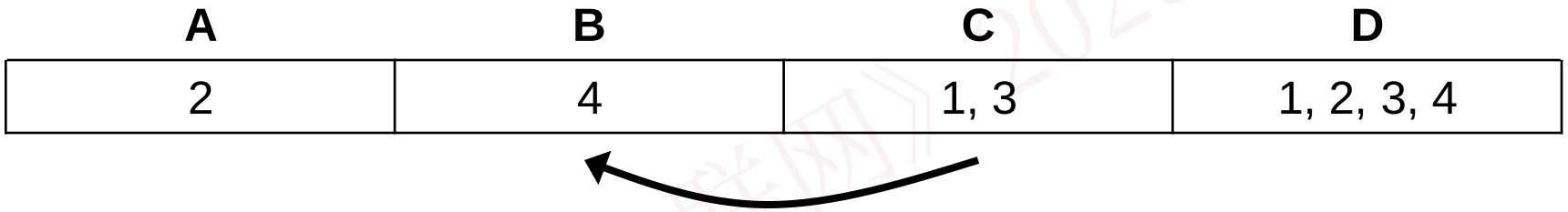
All arc A→B, A→C, A→D, B→A, B→C, B→D, C→A, C→B, C→D, D→A, D→B, D→C

queue C→A, C→B, C→D, D→A, D→B, D→C, A→B, A→C, B→C

浙大人工智能与物联网公2026夏

Enforcing Arc Consistency: 4-Queens



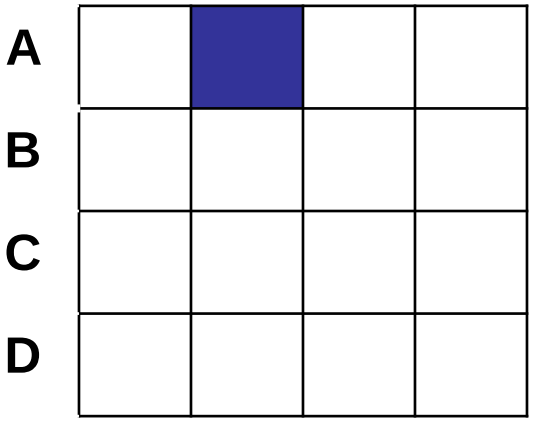


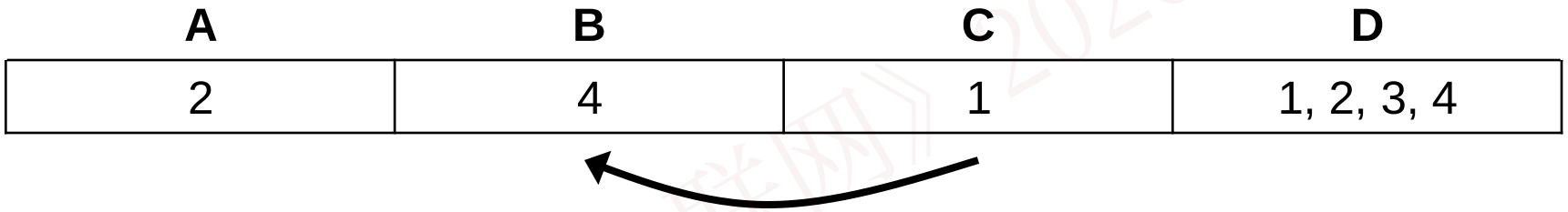
All arc A→B, A→C, A→D, B→A, B→C, B→D, C→A, C→B, C→D, D→A, D→B, D→C

queue C→B, C→D, D→A, D→B, D→C, A→B, A→C, B→C

浙大人工智能与物联网公2026夏

Enforcing Arc Consistency: 4-Queens



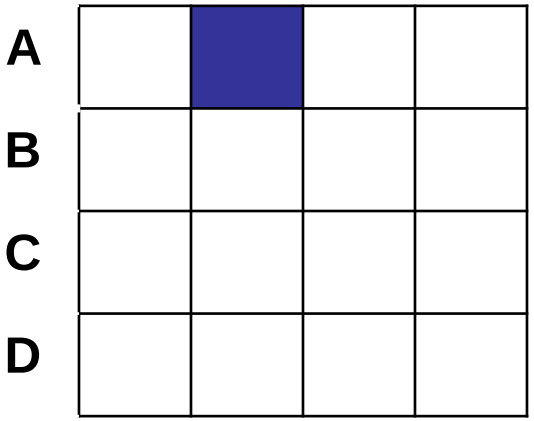


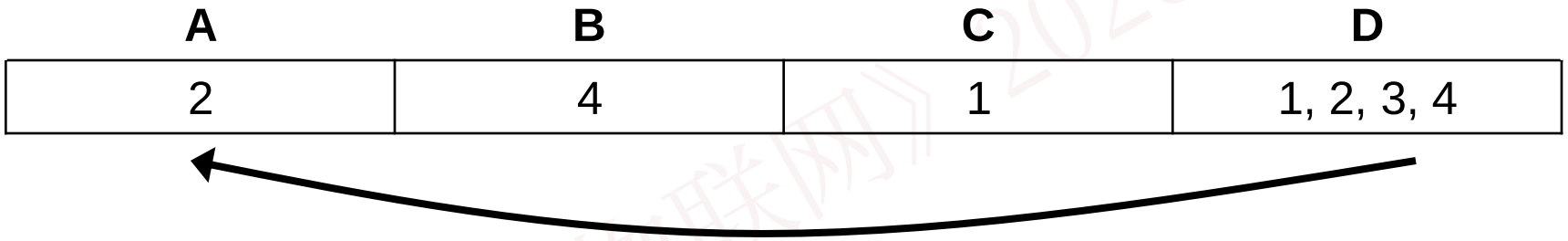
All arc A→B, A→C, A→D, B→A, B→C, B→D, C→A, C→B, C→D, D→A, D→B, D→C

queue C→B, C→D, D→A, D→B, D→C, A→B, A→C, B→C

浙大人工智能与物联网公 2026夏

Enforcing Arc Consistency: 4-Queens



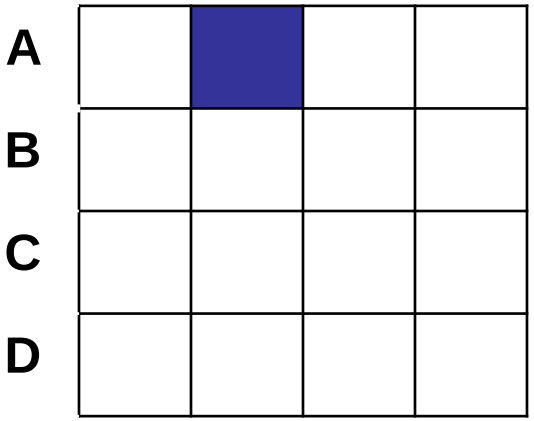


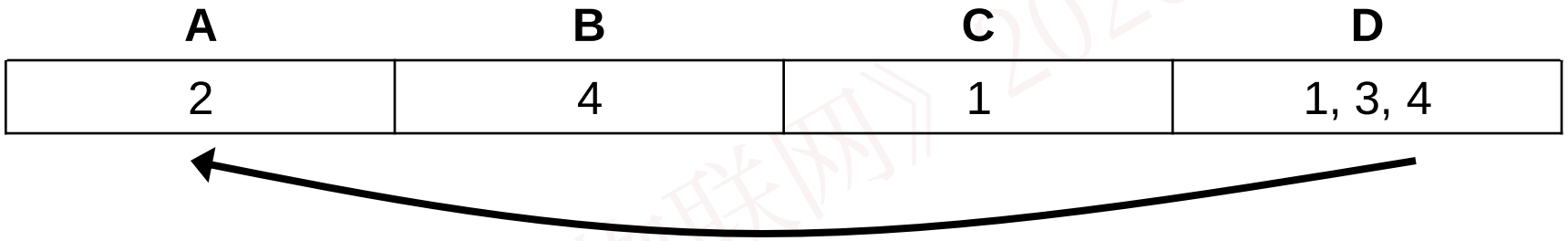
All arc A→B, A→C, A→D, B→A, B→C, B→D, C→A, C→B, C→D, D→A, D→B, D→C

queue D→A, D→B, D→C, A→B, A→C, B→C

浙大人工智能与物联网 2026夏

Enforcing Arc Consistency: 4-Queens



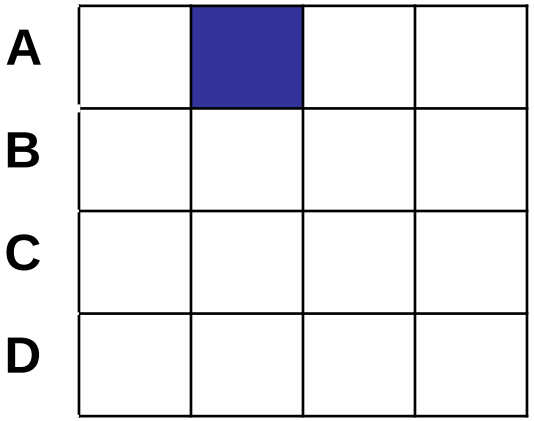


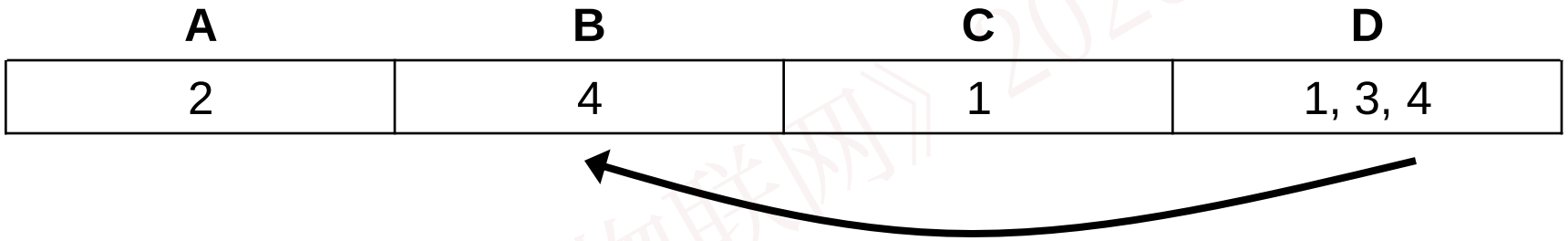
All arc A→B, A→C, A→D, B→A, B→C, B→D, C→A, C→B, C→D, D→A, D→B, D→C

queue D→A, D→B, D→C, A→B, A→C, B→C, A→D, B→D, C→D

浙大人工智能与物联网 2026夏

Enforcing Arc Consistency: 4-Queens



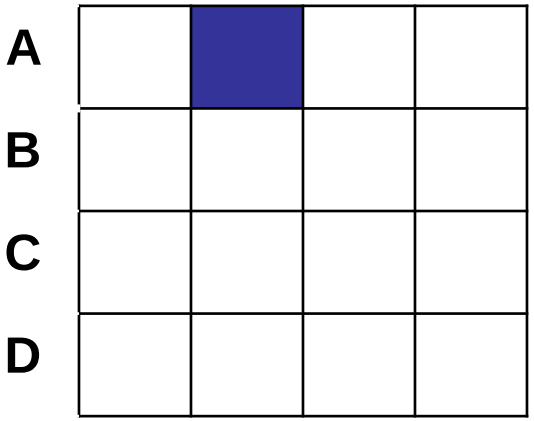


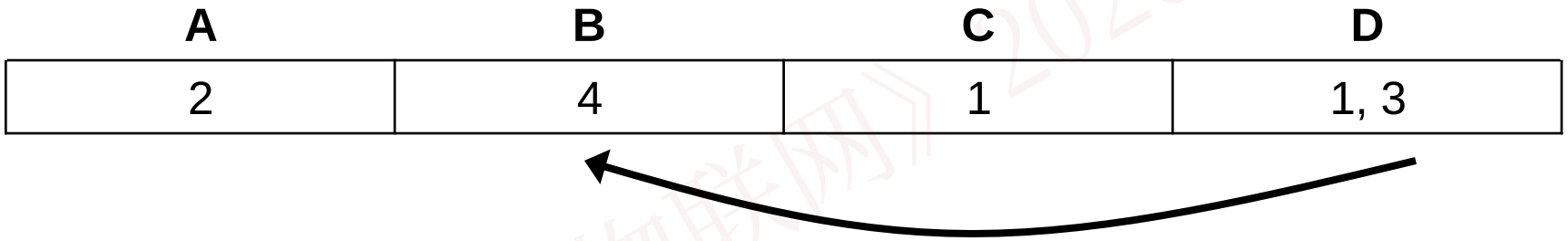
All arc A→B, A→C, A→D, B→A, B→C, B→D, C→A, C→B, C→D, D→A, D→B, D→C

queue D→B, D→C, A→B, A→C, B→C, A→D, B→D, C→D

浙大人工智能与物联网 2026夏

Enforcing Arc Consistency: 4-Queens



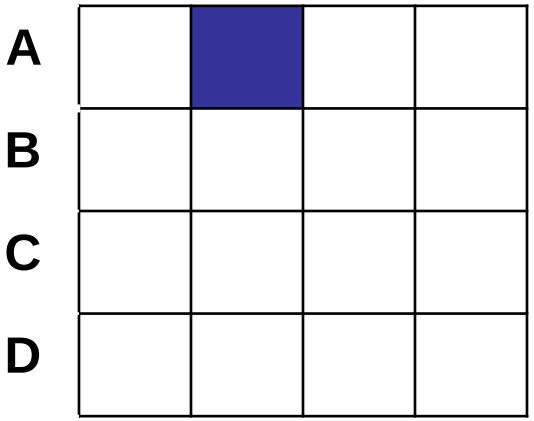


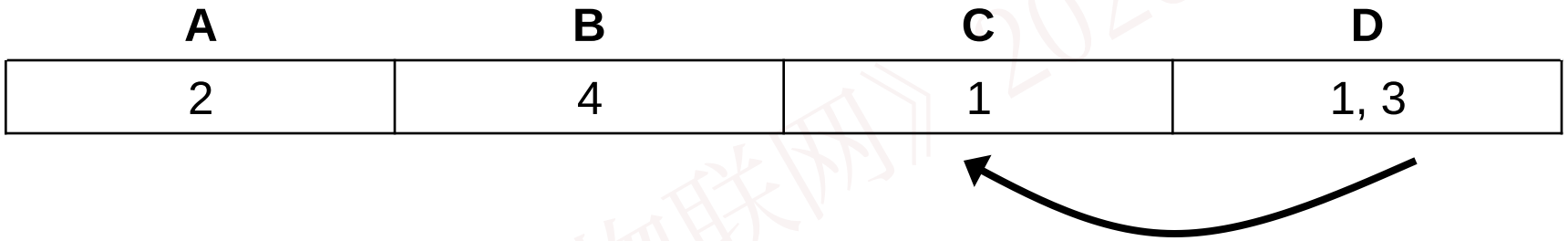
All arc A→B, A→C, A→D, B→A, B→C, B→D, C→A, C→B, C→D, D→A, D→B, D→C

queue D→B, D→C, A→B, A→C, B→C, A→D, B→D, C→D

浙大人工智能与物联网 2026夏

Enforcing Arc Consistency: 4-Queens



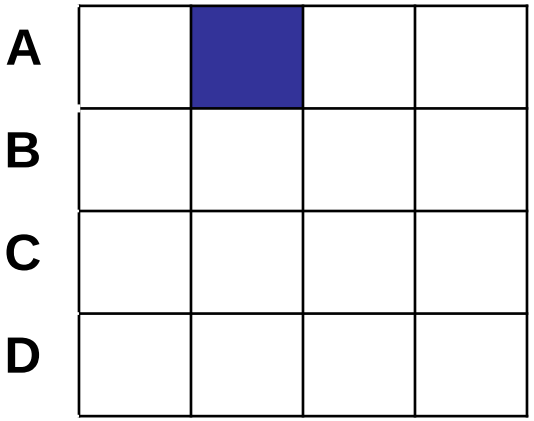


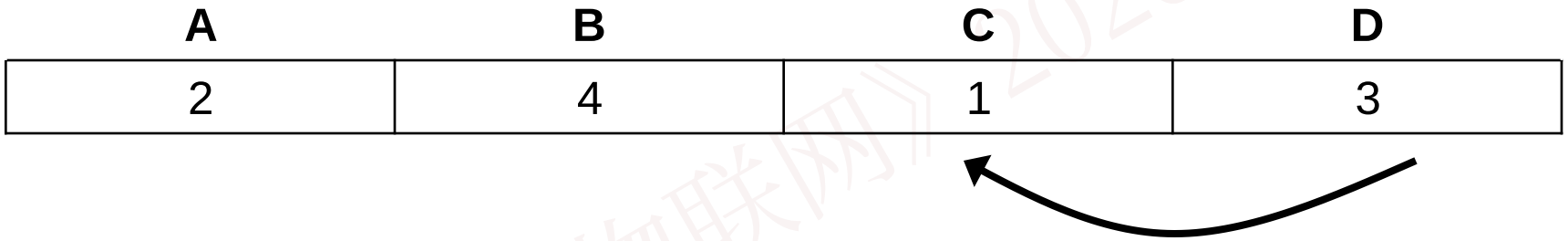
All arc A→B, A→C, A→D, B→A, B→C, B→D, C→A, C→B, C→D, D→A, D→B, D→C

queue D→C, A→B, A→C, B→C, A→D, B→D, C→D

浙大人工智能与物联网公2026夏

Enforcing Arc Consistency: 4-Queens



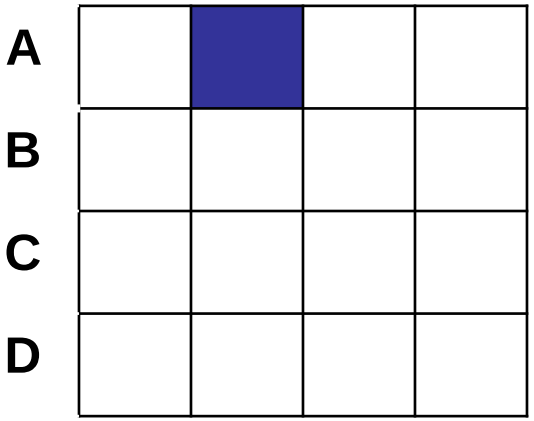


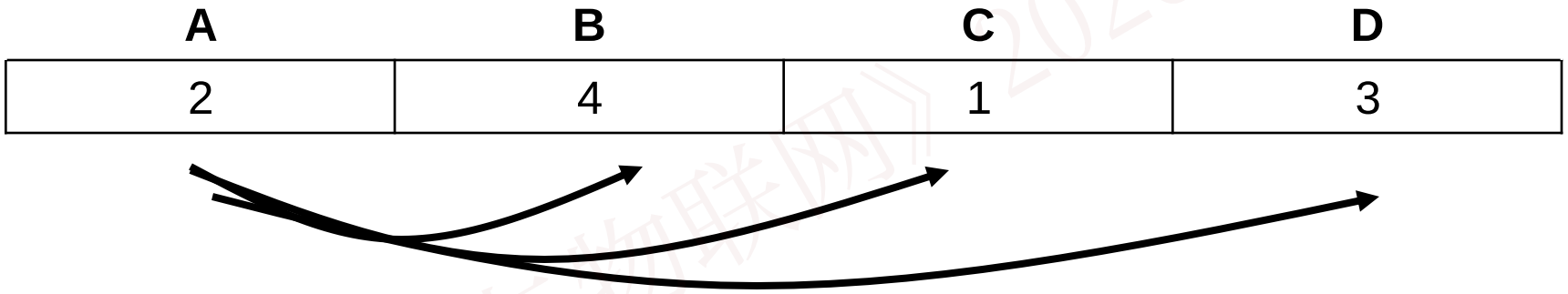
All arc A→B, A→C, A→D, B→A, B→C, B→D, C→A, C→B, C→D, D→A, D→B, D→C

queue D→C, A→B, A→C, B→C, A→D, B→D, C→D

浙大人工智能与物联网公2026夏

Enforcing Arc Consistency: 4-Queens





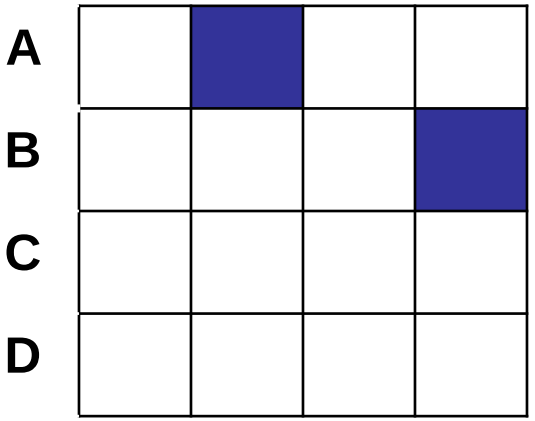
All arc A→B, A→C, A→D, B→A, B→C, B→D, C→A, C→B, C→D, D→A, D→B, D→C

queue A→B, A→C, B→C, A→D, B→D, C→D

Arc Consistency
Check finished!

浙大人工智能与物联网公 2026夏

Enforcing Arc Consistency: 4-Queens

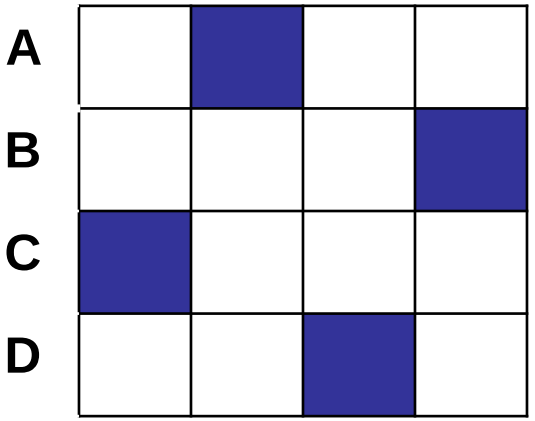


A	B	C	D
2	4	1	3

queue **A→B, A→C, A→D, B→A, B→C, B→D, C→A, C→B, C→D, D→A, D→B, D→C**

浙大《人工智能与物联网》2026夏

Enforcing Arc Consistency: 4-Queens



A	B	C	D
2	4	1	3

queue **A→B, A→C, A→D, B→A, B→C, B→D, C→A, C→B, C→D, D→A, D→B, D→C**

浙大《人工智能与物联网》2026夏